Assessment 4

Unit: Advanced Data Management
(PRT563)

Campus: ECP (Darwin)

Topic:

From Tables to Connections: Migrating The Industry School's Database from SQLite to Neo4j

Submitted By

Group: 2

Najmus Sakeeb (S393942)
Md Adnan Abir (S382198)
Cynthia Ebele Ikegbunam (S390803)
Chepngenoh Chepkwony(S399366)

Submitted To

Dr Md Rafiqul Islam

Lecturer, School of Engineering and Information Technology, CDU

Deadline: 16th Oct 2026

CHARLES DARWIN UNIVERSITY AUSTRALIA

Table of Contents

Executive Summary 1
1 Introduction 1
1.1 Context and research questions 1
1.2 Response to Assessment 2 feedback 2
1.3 Method overview 2
2 Graph Data Modelling (Task 1) 3
2.1 Conversion rules 4
2.2 The optimised model 4
3 Implementation (Task 2) 6
3.1 Import pipeline 6
3.2 Verification 6
4 Evaluating the Modelling Strategy 7
4.1 Experimental design 7
4.2 Results 7
5 Cypher Query Use Cases (Task 3) 8
5.1 Q1: Assessment conflict of interest (compliance manager) 8
5.2 Q2: Blue card compliance sweep (compliance manager) 10
5.3 Q3: Campus debt against the school-wide average (finance manager) 11
5.4 Q4: Prerequisite breaches and their root cause (curriculum lead) 13
6 Centrality Analysis (Task 4) 14
6.1 Use case and projection 14
6.2 Evaluation design 14
6.3 Results 15
6.4 Recommendation 16
7 Similarity Analysis (Task 5) 17
7.1 Use case and algorithms 17
7.2 Evaluation design 17
7.3 Results 17
7.4 Recommendation 18
8 Discussion and Limitations 20
9 Conclusion 21
References 22
Individual Contributions 22
Appendix A - Neo4j run evidence 23

List of Figures

Figure 1: Study design 2
Figure 2: Neo4j graph data model (Model B) 3
Figure 3: The five optimisations 5
Figure 4: The loaded graph as Neo4j reports it 6
Screenshot 1: Task 2 import totals in Neo4j Desktop (323 nodes, 633 relationships) 7
Figure 5: Modelling experiment 8
Screenshot 2: Q1 result in Neo4j Desktop 9
Figure 6: Q1 on the migrated data 10
Screenshot 3: Q2 result in Neo4j Desktop 11
Screenshot 4: Q3 result in Neo4j Desktop 12
Screenshot 5: Q4 result in Neo4j Desktop (all ten rows, shown in two parts) 14
Figure 7: Centrality algorithms compared 15
Figure 8: The delivery network 16
Screenshot 6: C9 key-person register in Neo4j Desktop 17
Figure 9: Similarity variants compared 18
Screenshot 7: S9 placement recommendations in Neo4j Desktop 19

List of Tables and Listings

Table 1: Assessment 2 feedback and the implemented changes 2
Table 2: How each EER construct was translated 4
Table 3: Migrated graph (Model B) against the source 7
Listing 1: Q1 (abridged; full version in queries.txt) 9
Listing 2: Q2 (abridged) 10
Table 4: Q2 result (reporting date 14 September 2026) 11
Listing 3: Q3 (abridged) 12
Table 5: Q3 result (school-wide average $2,553.00 over all five campuses) 12
Listing 4: Q4 (abridged) 13
Table 6: Key-person risk register (top five by betweenness) 16

Repository: https://github.com/adnanabiride/PRT563-ADVANCED-DATA-MANAGEMENT

Word count: Approx 5,580 words (excluding title page, contents, lists, code listings, references and appendix)

Executive Summary

The Industry School is a registered training organisation (RTO) in Queensland. Their administration is based on the SQLite database we developed in Assessment 2 (26 tables, 600 rows). The questions it has the most difficulty answering are the relationship questions - who teaches whose child, which trainer is the glue of the teaching network, which peers a student is like. In SQL, these require long joins or cannot be expressed as joins at all. We migrated the database into Neo4j and then asked whether an optimised graph model can provide measurable improvements over a direct translation.

Five optimisations, swapping IS_A hops for role labels, collapsing RESULT onto a TOOK relationship, flipping phone numbers, materialising invoice balances and an explicit NEXT_TERM chain shrank the graph from 569 to 323 nodes and a six-query workload from 1,777 to 1,374 logical accesses (-23%) with identical results. An ablation shows the value of the changes: not folding the workload costs 16% more, and not materialising 10%. Pivoted or not, the phone numbers don't matter.

The migration was validated and verified: all 3,143 checks on the 600 relational rows passed, and every script produced the expected results when run in Neo4j. The four Cypher use cases revealed one active assessment conflict of interest, four staff with blue cards expiring by the end of Term 4 (two of whom are rostered to teach in that term), two campuses with debt above the school-wide average (now calculated across all five campuses, as the Assessment 2 feedback required) and ten prerequisite breaches traced back to three root causes.

We evaluated seven centrality algorithms against an independent ground truth. Path-based measures correlate with network damage (Spearman ρ = 0.90-0.93), whereas degree, PageRank-type, and eigenvector measures do not (ρ = 0.29-0.49); we recommend betweenness, which is supported by harmonic centrality. Nine similarity variants were tested for placement matching. Similarity alone did no better than random, so we suggest it be used only inside a programme filter.

1 Introduction

1.1 Context and research questions

The Industry School is in Southeast Queensland. It is a senior-secondary and vocational training college with five campuses, all of which offer QCAA subjects and VET units. Assessment 2 consisted of drawing EER and UML models of the school's database; normalisation and the SQLite implementation were also completed. Assessment 4 involves migrating that SQLite database to a graph model in Neo4j, using the Graph Data Science (GDS) library for analytics. This assessment aims to answer the following research questions:

- **RQ1** Is an optimised graph model more efficient than a direct rule-based translation and which type of optimisation provides the gain?
- **RQ2** Can the graph model answer the stakeholders' finance, curriculum and compliance questions correctly and more directly than SQL?
- **RQ3** Based on independent evidence, which centrality and similarity algorithms should the school rely on for key-person risk and placement matching?

Following the Assessment 1 feedback on stakeholder interaction, this assessment frames each use case around a stakeholder: the compliance manager, finance manager, curriculum lead and placement officer.

1.2 Response to Assessment 2 feedback

Feedback	Corrections made	Where the changes were made
The debt query did not include campuses with zero debt in the school-wide average	The average now includes all campuses, even those with zero balances (SQL outer joins; Cypher OPTIONAL MATCH). It is 2,553.00 either way today, but if one campus owed nothing it would be 2,502.00 instead of an inflated 3,127.50	Q3; `a2_feedback_fixes.sql`
Billing-account category membership not enforced	The modified SQL code now contains three UNIQUE indexes and six triggers. The graph model adds one uniqueness constraint on a relationship property plus a holder-type check	`import.txt` s.1, 8.4, 8.8
UML omitted academic and placement classes; invoice composition reversed	UML redrawn: 25 classes plus an «interface» for the billing category; filled diamond at the Invoice (whole) end	`A2_revised_UML.png`
Presentation too long	No presentation in A4; report held to the word limit, detail moved to the repository	Repository

Table 1: *Assessment 2 feedback and the implemented changes*

1.3 Method overview

Figure 1 summarises the study. The relational data was exported table by table and then translated into two graph models. The graphs were loaded, verified cell by cell, and run through our query, experiment, and analytics tasks. All Cypher was checked against Neo4j's own parser (`@neo4j-cypher/language-support`). The expected results for every script were generated by an independent in-memory replica of the graph and are available in the repository, so each result can be rechecked in Neo4j Desktop. All scripts were then run on Neo4j Desktop 2.2.1 (Neo4j 2026.09.0, Cypher 25) with GDS 2026.09.0 on 14 October 2026. Every count, query row, and centrality ranking matched the expected results; the few differences in detail are reported where they arise, and screenshots of every step are in the repository (supplementary/run_evidence/ and Appendix A).

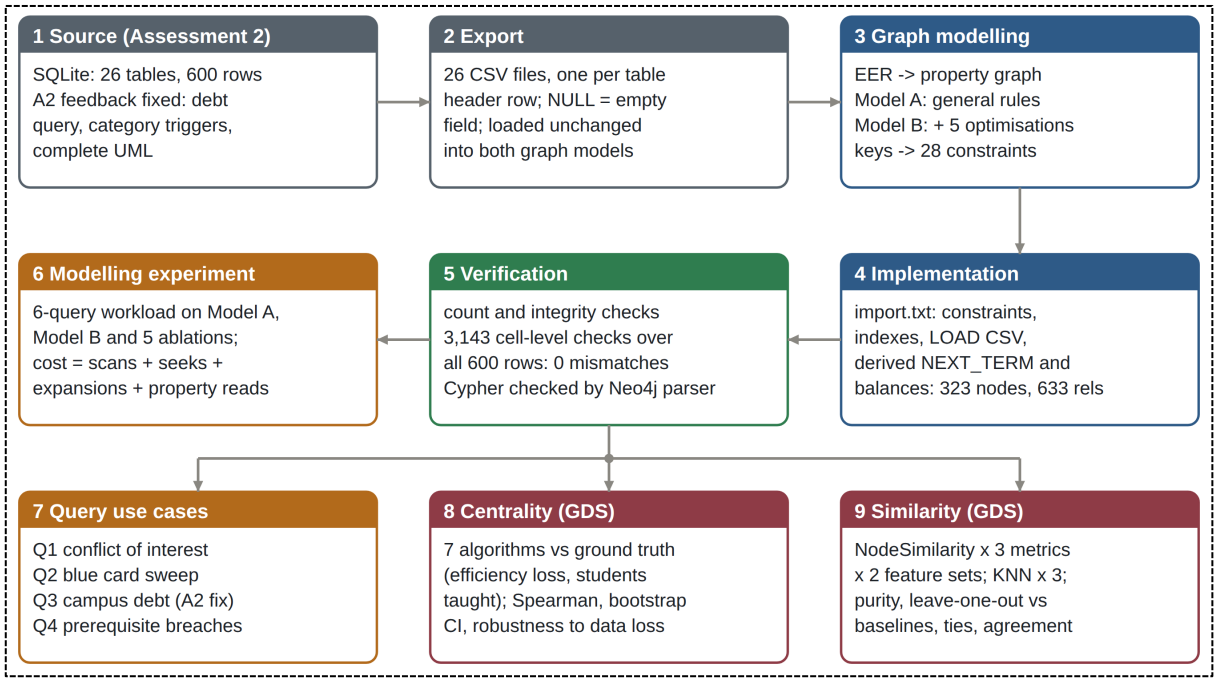


Figure 1: *Study design*

Steps 1-2 export the Assessment 2 database, steps 3-5 model, load and verify it and steps 6-9 are the modelling experiment, the query use cases and the two analytics tasks.

2 Graph Data Modelling (Task 1)

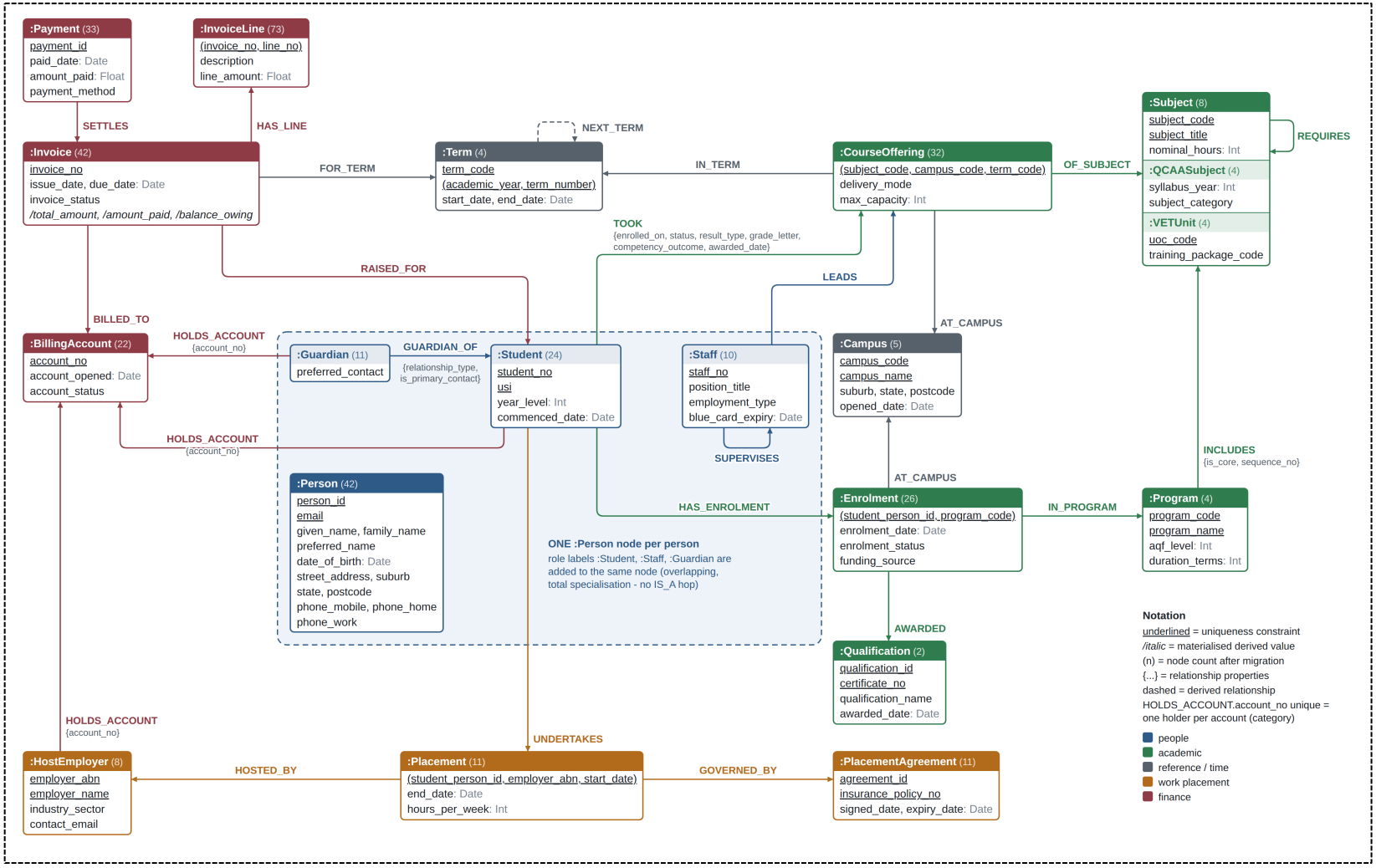


Figure 2: *Neo4j graph data model (Model B)*

Boxes are node labels with their properties; the dashed frame is one :Person node carrying up to three role labels; arrows are relationship types. Full-size SVG in the repository: figures/figure2_graph_model.svg.

2.1 Conversion rules

The resulting graph model is shown in Figure 2 (Model B). Table 2 shows the general mapping rules from EER to property graph for each construct (Robinson et al., 2015; Lemahieu et al., 2018). Model A stops at the general mapping rules and is the baseline; Model B adds five optimisations (O1-O5) and typed values. Both models keep all relational keys. In Model B the 15 entity primary keys and 12 of the 13 UNIQUE constraints become uniqueness constraints (composite where the key is composite). The remaining UNIQUE constraint, one qualification per enrolment, becomes a cardinality rule checked by validation 8.7. Finally, the key of each junction table is the identity of the relationship connecting a pair of nodes.

EER / relational construct	Instance	Model A (general rules)	Model B (optimised)
Strong entity, primary key	Campus, Term, Program ...	label + uniqueness constraint	same
Overlapping total specialisation	Person: Student, Staff, Guardian	subtype node + `IS_A`	extra labels on one node (O1)
Disjoint total specialisation	Subject: QCAA, VET	subtype node + `IS_A`; discriminator kept	labels; discriminator dropped (O1)
Category (union type)	BillingAccount	`HOLDS_ACCOUNT` from 3 superclasses	same + unique `account_no` on the relationship
M:N with attributes; recursive	Guardianship, ProgramSubject; prerequisites, supervision	relationship with properties	same
Ternary relationship / aggregation	CourseOffering, Enrolment, Placement	reified node + composite key	same
Associative entity + 1:1 dependant	SubjectEnrolment + Result	two nodes, three relationships	one `TOOK` relationship (O2)
Weak entity	InvoiceLine	node + `HAS_LINE` + (invoice_no, line_no)	same
Multi-valued attribute	PersonPhone	`PhoneNumber` nodes	three typed properties (O3)
Derived attribute	/total_amount (view)	computed at query time	materialised on Invoice (O4)
Implicit order	Term (year, number)	sorted at query time	`NEXT_TERM` chain (O5)
Foreign-key column	e.g. Invoice.term_code	relationship + copied property	relationship only
Data types	dates, money, flags	strings	Date, Float, Boolean

Table 2: *How each EER construct was translated*

2.2 The optimised model

Model B has 20 labels, 22 relationship types, 323 nodes and 633 relationships. It is shaped by four decisions.

People form one node: The specialisation of PERSON overlaps: Priya Raman can be a student, a staff member, and a guardian at the same time - so the roles become labels on a single :Person node. In the relational model, it took four tables and three joins to put her back together. Here we just MATCH (p:Staff&Guardian). The same decision also retires the subject_kind discriminator column and labels SUBJECT's disjoint specialisation into :QCAASubject and :VETUnit.

Binary facts become relationships; n-ary facts become nodes. We reify the ternary COURSE_OFFERING (subject × campus × term), the aggregations ENROLMENT and PLACEMENT and the weak entity INVOICE_LINE as nodes, whose composite keys are enforced by composite uniqueness constraints, because a property-graph relationship always connects exactly two nodes. PLACEMENT_AGREEMENT remains its own node because a single agreement covers multiple placements. SUBJECT_ENROLMENT, however, is binary, and RESULT is one-to-one dependent on it, so they are both reduced to a single TOOK relationship carrying six properties.

The category rule is declarative: BILLING_ACCOUNT is the union of STUDENT, GUARDIAN, and HOST_EMPLOYER, and each account must be a member of exactly one of them (Elmasri & Navathe, 2016). In Neo4j 5.7+ Community Edition, a unique copy of account no in HOLDS_ACCOUNT prevents any second holder of any type; SQLite required three UNIQUE indexes and six triggers for the same rule.

Structures that query walk are stored: NEXT_TERM cuts "the following term" down to one hop and the invoice balance is materialised because finance queries read it far more often than payments update it.

2.3 Optimisations and rejected alternatives

Figure 3 compares each optimisation with the pattern replaced. Three other options were turned down. Storing phone numbers as a list property would lose the phone type. The pivot is safe because each person has at most one number of each type. Collapsing INVOICE_LINE to a list on Invoice would lose the weak entity identity and the ability to query at the line level. The idea of storing a student GPA was rejected because grades change every term and there was no use case for it. Each optimisation thus trades off something - schema flexibility for O3, write work and possible staleness for O4, and Section 4 considers whether the benefit outweighs the cost.

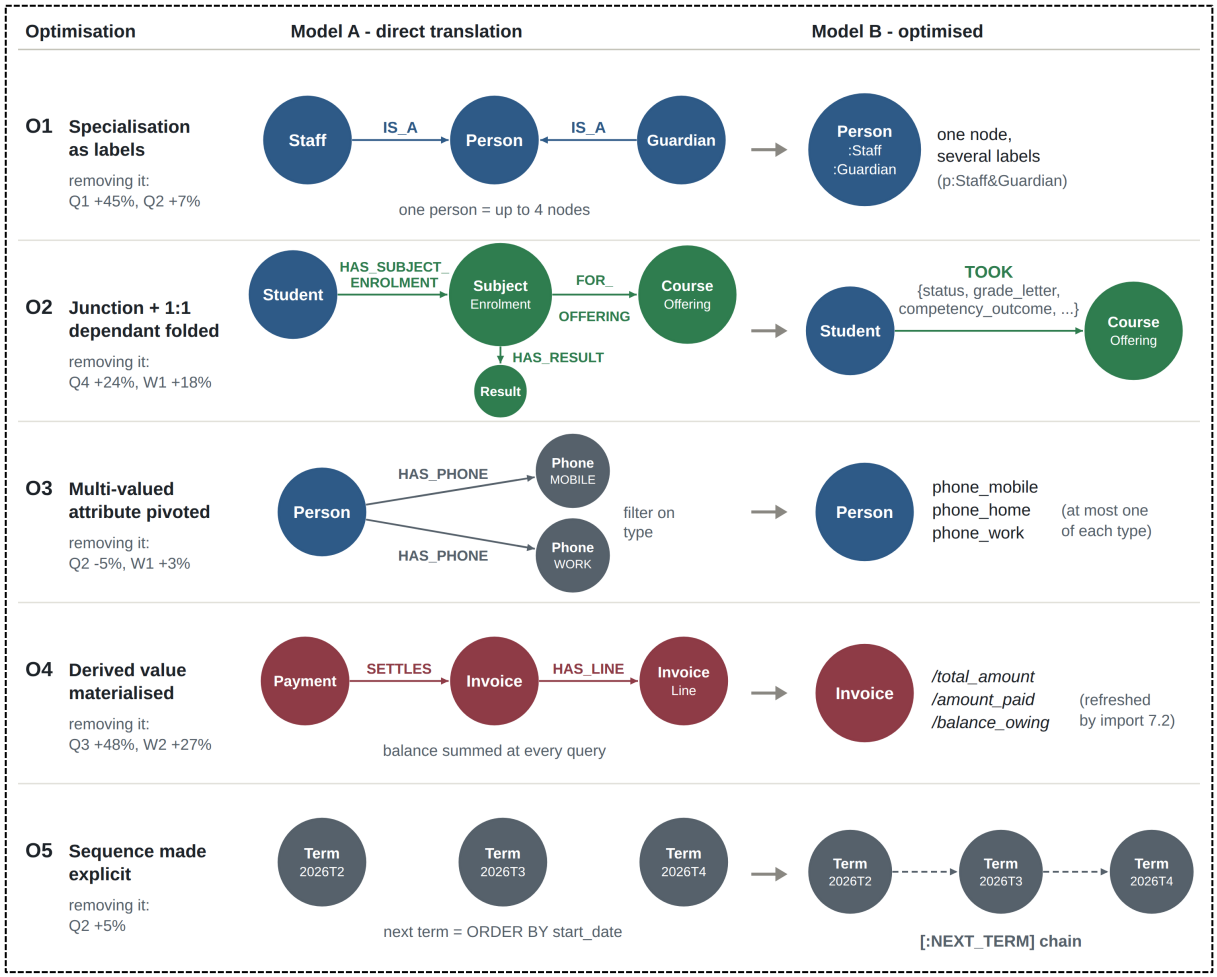


Figure 3: *The five optimisations*

Model A pattern (left) and Model B pattern (right); under each name, the measured cost of removing that optimisation (Section 4).

3 Implementation (Task 2)

3.1 Import pipeline

The import.txt file imports the 26 CSV documents in 8 sections. First it creates 28 uniqueness constraints so that each MERGE and relationship lookup is an index seek. It also creates 5 query-driven indexes, two of them on relationship properties (TOOK.status, TOOK.competency_outcome) so that queries like "all current enrolments" can start from the relationship itself (Neo4j, Inc., 2026a). Then reference nodes, people and reified nodes are merged, relationships created from foreign keys and junction tables and the derived NEXT_TERM chain and invoice balances are calculated. Each statement is idempotent (MERGE), converts each value to the right type (date(), toInteger(), toFloat()) and guards each empty field with CASE WHEN row.x <> '' THEN ... END so missing values are absent, not empty strings. The materialised balances can be updated after new payments by running Section 7.2 once again. For production volumes, the same statements can be wrapped in CALL { ... } IN TRANSACTIONS to commit in batches.

3.2 Verification

This section of import.txt tests that Neo4j imports correctly. It checks node, relationship and row counts (8.1-8.3), category correctness (8.4), specialisation totality and disjointness (8.5-8.6), and a structural-integrity query (8.7) that replaces the NOT NULL and cardinality constraints the Community Edition cannot say. Statement 8.8 behaved as expected: Neo4j refused to accept a second holder for account ACC-1001 because of an error involving the uniqueness constraint (GQLSTATUS 22N79). A reconciliation script iterated over each column of each relational row and checked it against the graph, treating each as a property, a label, or a relationship. It ran 3,143 checks, all of which were correct. So the migration is complete: nothing was lost, and only two things are stored differently from SQL - the discriminator is now a label and the phone table is now three properties on the person. Figure 4 shows the schema Neo4j provides for the loaded graph. It comprises the 20 labels and 22 relationship types depicted in Figure 2.

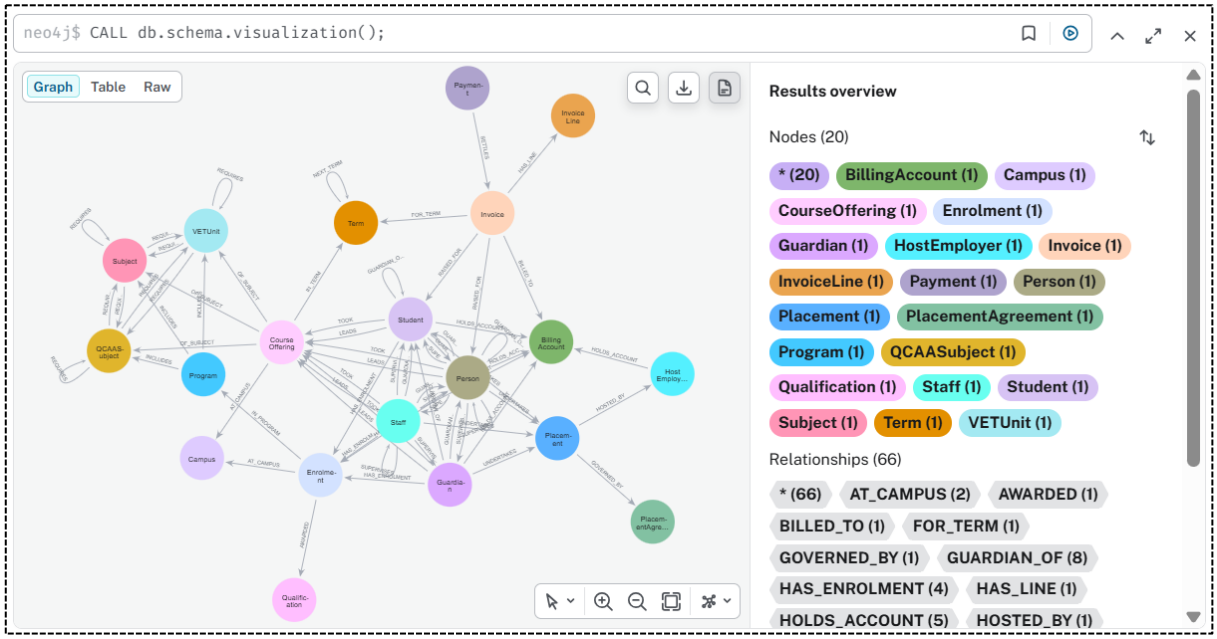
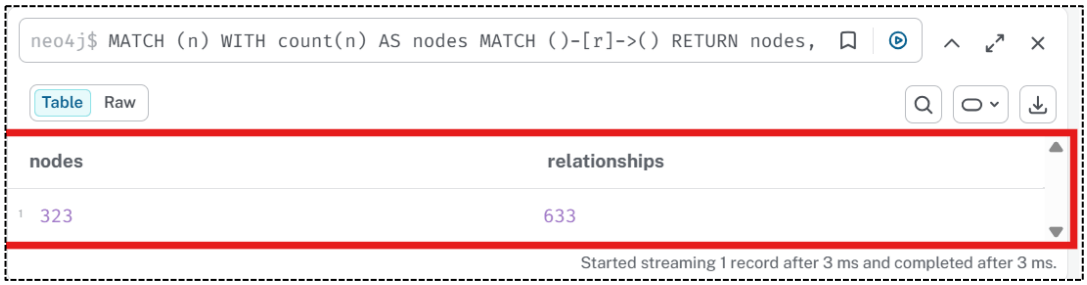


Figure 4: *The loaded graph as Neo4j reports it*

Output of CALL db.schema.visualization() after import.txt. The overview lists 66 relationship entries because each type is drawn once per pair of labels it joins. Neo4j's SVG export: figures/figure4_neo4j_schema_export.svg.

Measure	Relational source	Model B	Model A (baseline)
Tables / labels	26 tables	20 labels	23 labels
Rows / nodes	600 rows	323 nodes	569 nodes
Relationships	-	633 (22 types)	876 (25 types)
Stored values	-	2,384 properties	3,162 properties
Reconciliation	600 rows	3,143 checks, 0 mismatches	-

Table 3: *Migrated graph (Model B) against the source*



Screenshot 1: *Task 2 import totals in Neo4j Desktop (323 nodes, 633 relationships)*

4 Evaluating the Modelling Strategy

4.1 Experimental design

To address RQ1, we constructed both models from the same CSV data files through a single parameterised loader and performed the same six-query workload on each: the four use cases of Section 5 plus two front-office lookups (W1, a student profile; W2, an account statement), averaged over all 24 students and 22 accounts. We measured cost in logical accesses (label scans, index seeks, relationship expansions, and property reads), counted by an instrumented replica that records the pattern order of each query. This is a database-independent metric, analogous to Neo4j's notion of database hits, but does not include caching and planner effects. experiment_profile.txt Reruns the same workload with PROFILE. Finally, a leave-one-out ablation was performed on Model B, removing each optimisation in turn. The same results were obtained on all six queries for each variant, so cost differences are purely a function of structure.

4.2 Results

Model B needed fewer accesses to respond to each query (Figure 5a). Total workload reduced from 1,777 to 1,374 accesses, a 23% reduction. Relationship expansions were down 33% and distinct nodes touched by 49%. We also removed 201 conversions from strings to dates or numbers. The conflict-of-interest query (−37%), where Model A does more than one IS_A hop, and the debt query (−33%) saved the most. The debt query prompts Model A to resume all of its invoices and payments.

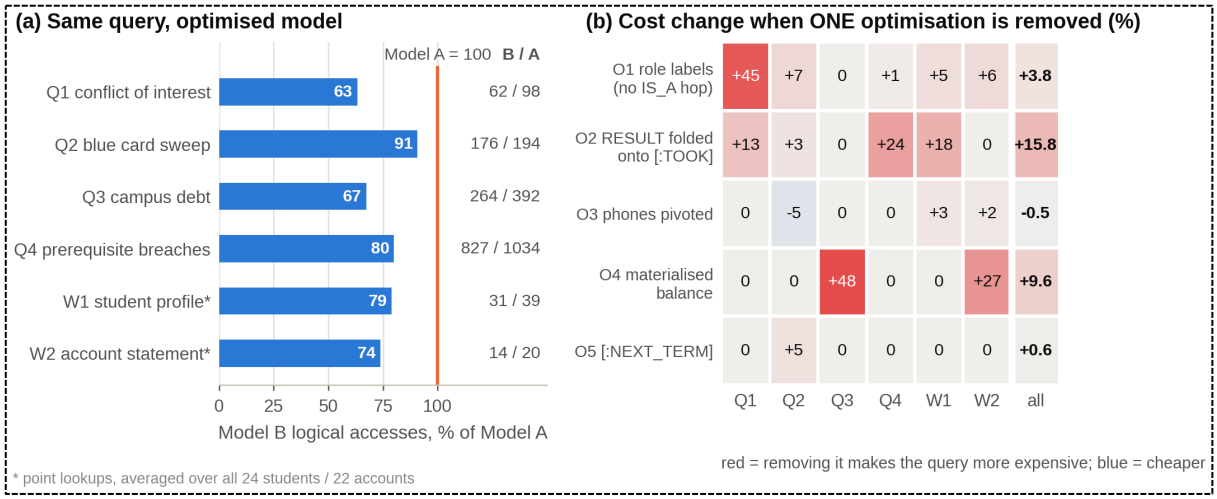


Figure 5: *Modelling experiment*

(a) Model B cost as a percentage of Model A for each query. (b) Change in cost when one optimisation is removed from Model B.

The ablation (Figure 5b) clearly separates the optimisations. Crucially, overall, folding RESULT onto TOOK (+16% workload cost without, +24% on Q4). Materialisation wins where money is pooled (+48% on Q3). Role labels help for role-centric queries (+45% on Q1). NEXT_TERM does not gain much at four terms, but its advantage improves with longer histories. Phone pivoting is cost neutral (−0.4%). Reading three missing properties costs about the same as expanding zero HAS_PHONE relationships, so the benefit is just simpler queries and 27 fewer nodes. The sum of the five effects is very close to the cost of Model A (+29%).

These results are consistent with the property-graph literature: storing a fact where queries walk and removing hops that do not carry information pays off in proportion to the frequency with which the hop is walked (Robinson et al., 2015). They also demonstrate that an optimisation is not free: O4 adds write work to each payment - so it is the ablation, not intuition, that decides whether to keep it.

5 Cypher Query Use Cases (Task 3)

The four queries in `queries.txt` increase in difficulty and use a fixed reporting date of 14 September 2026 (final week of Term 3) so that their results can be replicated. Each is framed for a specific stakeholder.

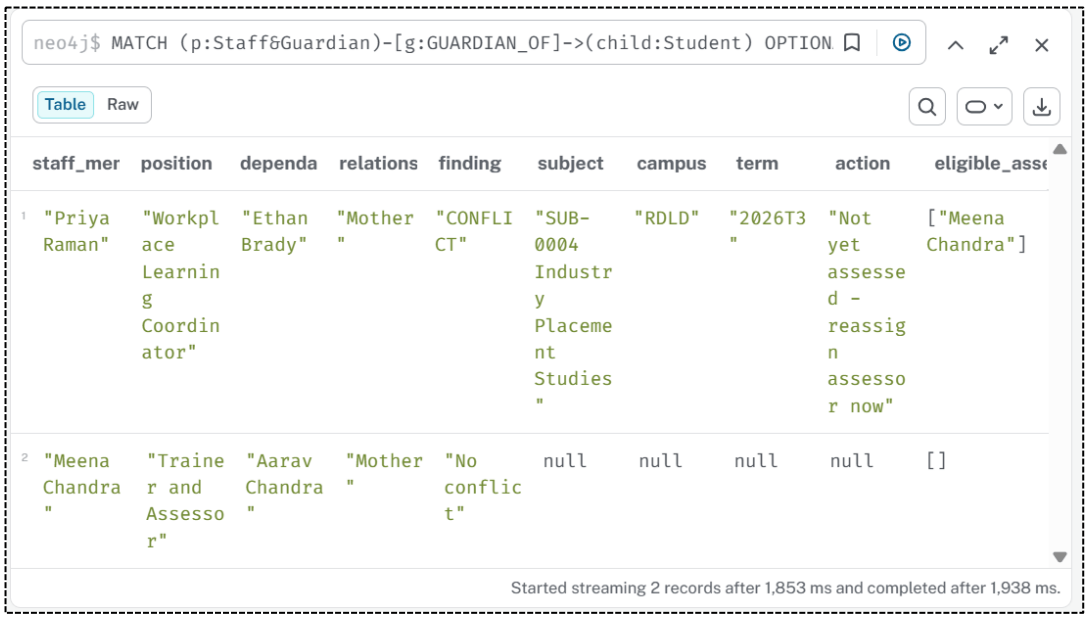
5.1 Q1: Assessment conflict of interest (compliance manager)

Assessment should be fair (Australian Skills Quality Authority [ASQA], 2025). Roles can be blurred: a trainer can be a parent. Q1 finds staff scheduled to assess their own children; the pattern completes a cycle on the same node p, and a COLLECT subquery finds eligible alternative assessors.

```
MATCH (p:Staff&Guardian)-[g:GUARDIAN_OF]->(child:Student)
OPTIONAL MATCH (child)-[t:TOOK]->(o:CourseOffering)<-[:LEADS]-(p),
               (o)-[:OF_SUBJECT]->(subj:Subject), (o)-[:IN_TERM]->(term:Term)
RETURN p.preferred_name + ' ' + p.family_name AS staff_member,
       child.preferred_name + ' ' + child.family_name AS dependant,
       CASE WHEN o IS NULL THEN 'No conflict' ELSE 'CONFLICT' END AS finding,
       subj.subject_code AS subject, o.campus_code AS campus,
       term.term_code AS term,
       CASE WHEN o IS NULL THEN null
            WHEN t.result_type IS NULL
                 THEN 'Not yet assessed - reassign assessor now'
            ELSE 'Result awarded ' + toString(t.awarded_date) + ' - re-mark'
       END AS action,
       COLLECT { MATCH (alt:Staff)-[:LEADS]->(:CourseOffering)-[:OF_SUBJECT]->(subj)
                 WHERE alt <> p
                   AND NOT EXISTS { (alt)-[:GUARDIAN_OF]->(child) }
                   AND alt.blue_card_expiry >= term.end_date
                 RETURN DISTINCT alt.preferred_name + ' ' + alt.family_name
       } AS eligible_assessors
ORDER BY finding, staff_member;
```

Listing 1: *Q1 (abridged; full version in queries.txt)*

Result: Two members of staff are also guardians. Priya Raman is scheduled to assess her son Ethan in SUB-0004 (Redlands, Term 3); the result has not been entered yet so the assessor can still be changed, and Meena Chandra is eligible. Meena does not teach any of the classes in which her own child is enrolled (Figure 6).



Screenshot 2: *Q1 result in Neo4j Desktop*

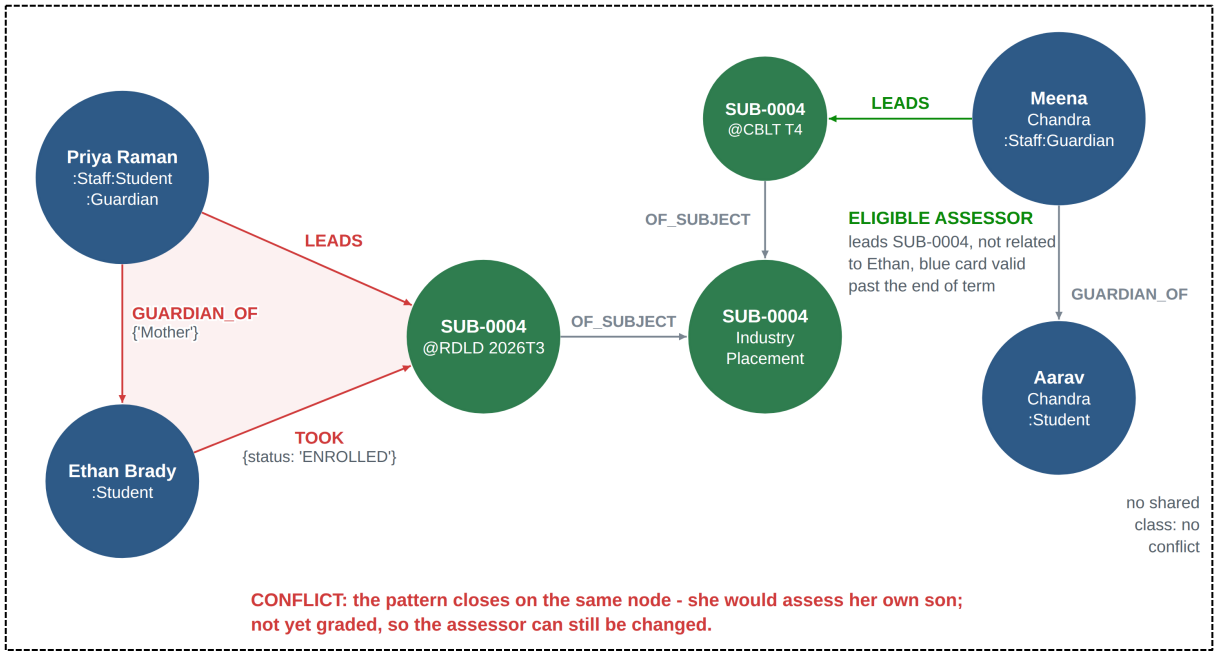


Figure 6: *Q1 on the migrated data*
The conflict-of-interest cycle (red) and the eligible assessor (green).

5.2 Q2: Blue card compliance sweep (compliance manager)

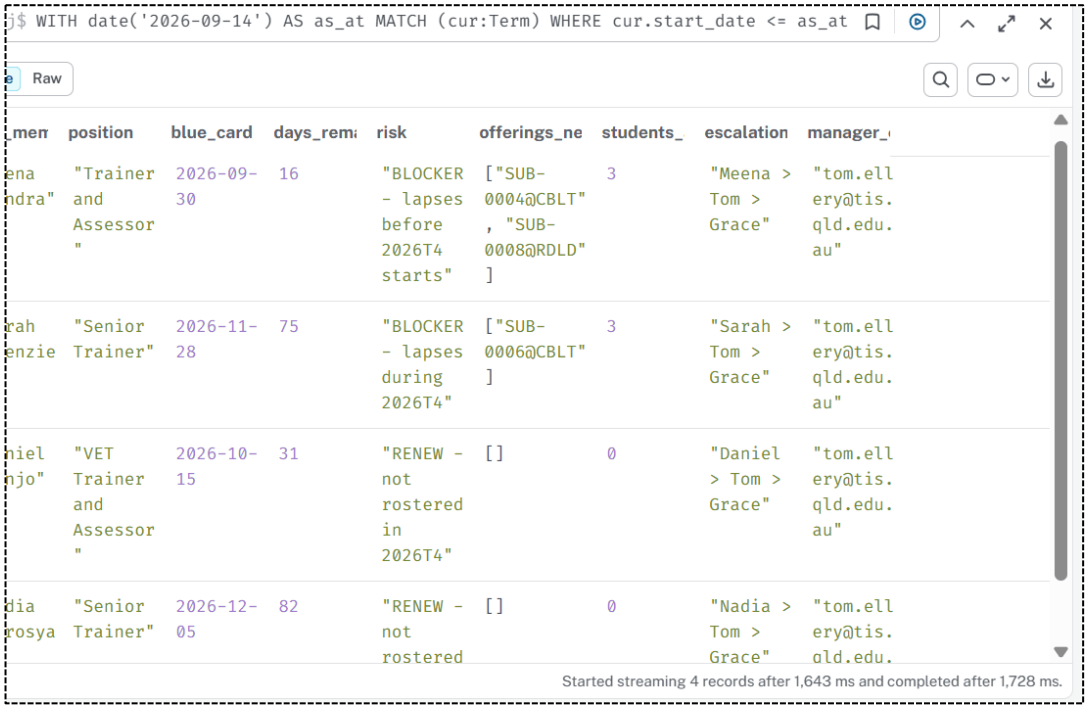
Queensland requires a current blue card to work with children (*Working with Children (Risk Management and Screening) Act 2000* (Qld)). Before the Term 4 roster is published, Q2 lists all staff whose card will expire before the end of next term, reached via one `NEXT_TERM` hop. It then shows whether they are rostered, how many students are affected and who to escalate to up the `SUPERVISES` hierarchy.

```
WITH date('2026-09-14') AS as_at
MATCH (cur:Term) WHERE cur.start_date <= as_at <= cur.end_date
MATCH (cur)-[:NEXT_TERM]->(nxt:Term)
MATCH (s:Staff) WHERE s.blue_card_expiry <= nxt.end_date
MATCH chain = (top:Staff)-[:SUPERVISES*0..]->(s)
WHERE NOT EXISTS { ()-[:SUPERVISES]->(top) }
WITH as_at, nxt, s, reverse(nodes(chain)) AS upward,
     COLLECT { MATCH (s)-[:LEADS]->(o:CourseOffering)-[:IN_TERM]->(nxt)
               RETURN o.subject_code + '@' + o.campus_code } AS rostered,
     COLLECT { MATCH (s)-[:LEADS]->(o:CourseOffering)-[:IN_TERM]->(nxt),
                     (o)<-[e:TOOK]-(st:Student)
               WHERE e.status = 'ENROLLED'
               RETURN DISTINCT st.person_id } AS students
RETURN s.preferred_name + ' ' + s.family_name AS staff_member,
       duration.inDays(as_at, s.blue_card_expiry).days AS days_remaining,
       CASE WHEN size(rostered) = 0 THEN 'RENEW - not rostered in ' + nxt.term_code
            WHEN s.blue_card_expiry < nxt.start_date
                 THEN 'BLOCKER - lapses before ' + nxt.term_code + ' starts'
            ELSE 'BLOCKER - lapses during ' + nxt.term_code END AS risk,
       rostered, size(students) AS students_affected,
       reduce(c = '', x IN upward |
              c + CASE WHEN c = '' THEN '' ELSE ' > ' END
                + x.preferred_name) AS escalation,
       coalesce(upward[1].phone_mobile, upward[1].phone_work,
                upward[1].email) AS contact
ORDER BY risk, days_remaining;
```

Listing 2: *Q2 (abridged)*

Staff member	Card lapses	Days	Risk	Term 4 classes	Students	Escalation
Meena Chandra	30 Sep 2026	16	BLOCKER - lapses before 2026T4 starts	SUB-0004@CBLT, SUB-0008@RDLD	3	Meena > Tom > Grace
Sarah McKenzie	28 Nov 2026	75	BLOCKER - lapses during 2026T4	SUB-0006@CBLT	3	Sarah > Tom > Grace
Daniel Okonjo	15 Oct 2026	31	RENEW - not rostered in 2026T4	-	0	Daniel > Tom > Grace
Nadia Petrosyan	5 Dec 2026	82	RENEW - not rostered in 2026T4	-	0	Nadia > Tom > Grace

Table 4: *Q2 result (reporting date 14 September 2026)*



Screenshot 3: *Q2 result in Neo4j Desktop*

Tom Ellery has no phone on record so `coalesce()` returns his email. In SQL the same answer needs a recursive CTE for the chain, date arithmetic for "next term" and a pivot for the contact.

5.3 Q3: Campus debt against the school-wide average (finance manager)

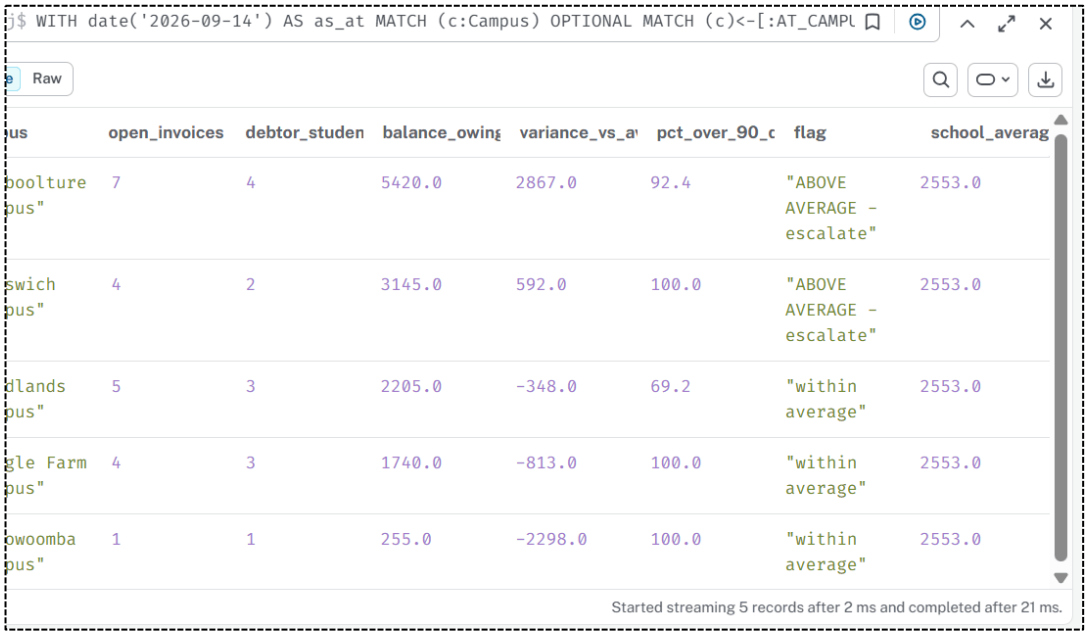
Q3 is the revised Assessment 2 debt query. The `WHERE` belongs to the `OPTIONAL MATCH` so a campus with no open invoices carries on with an empty list and contributes 0 to the average; the materialised balance means the query does not need to re-total every invoice from its lines and payments.

```
WITH date('2026-09-14') AS as_at
MATCH (c:Campus)
OPTIONAL MATCH (c)<-[:AT_CAMPUS]-(:Enrolment {enrolment_status: 'ACTIVE'})
               <-[:HAS_ENROLMENT]-(s:Student)<-[:RAISED_FOR]-(i:Invoice)
WHERE i.balance_owing > 0
WITH as_at, c, collect(DISTINCT i) AS open_invoices, count(DISTINCT s) AS debtors
WITH as_at, c, open_invoices, debtors,
     reduce(t = 0.0, x IN open_invoices | t + x.balance_owing) AS debt,
     reduce(a = 0.0, x IN open_invoices |
            a + CASE WHEN duration.inDays(x.due_date, as_at).days > 90
                     THEN x.balance_owing ELSE 0.0 END) AS aged_90
WITH collect({campus: c.campus_name, invoices: size(open_invoices),
              debtors: debtors, debt: debt, aged_90: aged_90}) AS campuses
WITH campuses,
     reduce(t = 0.0, x IN campuses | t + x.debt) / size(campuses) AS school_average
UNWIND campuses AS row
RETURN row.campus AS campus, row.invoices AS open_invoices, row.debtors AS debtors,
       round(row.debt, 2) AS balance_owing,
       round(row.debt - school_average, 2) AS variance_vs_average,
       CASE WHEN row.debt = 0 THEN 0.0
            ELSE round(100.0 * row.aged_90 / row.debt, 1) END AS pct_over_90_days,
       CASE WHEN row.debt > school_average THEN 'ABOVE AVERAGE - escalate'
            ELSE 'within average' END AS flag
ORDER BY balance_owing DESC;
```

Listing 3: *Q3 (abridged)*

Campus	Open invoices	Debtors	Owing ($)	vs average ($)	> 90 days	Flag
Caboolture	7	4	5,420.00	+2,867.00	92.4%	ABOVE AVERAGE
Ipswich	4	2	3,145.00	+592.00	100.0%	ABOVE AVERAGE
Redlands	5	3	2,205.00	−348.00	69.2%	within average
Eagle Farm	4	3	1,740.00	−813.00	100.0%	within average
Toowoomba	1	1	255.00	−2,298.00	100.0%	within average

Table 5: *Q3 result (school-wide average $2,553.00 over all five campuses)*



Screenshot 4: *Q3 result in Neo4j Desktop*

The totals equal the corrected SQL query exactly - a cross-model check that the migration preserved the financial data.

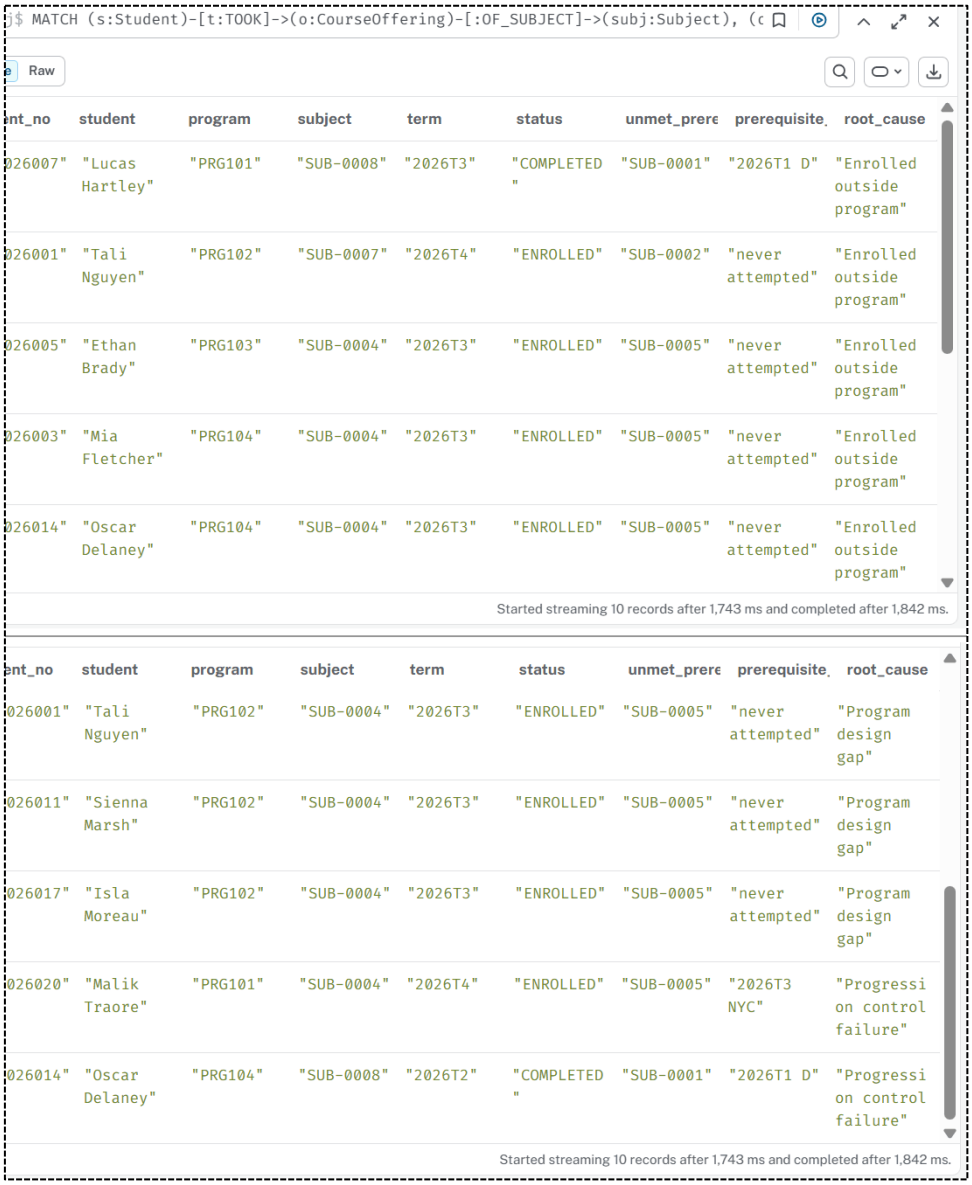
5.4 Q4: Prerequisite breaches and their root cause (curriculum lead)

Q4 matches every enrolment whose prerequisites, at any level, were not passed in a previous term and labels the reason so it can be corrected at the source. The query uses a quantified relationship (-[:REQUIRES]->+), a pattern comprehension with a nested comprehension, the list predicate `none()` and `EXISTS` subqueries.

```
MATCH (s:Student)-[t:TOOK]->(o:CourseOffering)-[:OF_SUBJECT]->(subj:Subject),
      (o)-[:IN_TERM]->(term:Term)
WHERE t.status <> 'WITHDRAWN'
MATCH (subj)-[:REQUIRES]->+(pre:Subject)
WITH DISTINCT s, t, subj, term, pre
WITH s, t, subj, term, pre,
     [(s)-[t2:TOOK]->(o2:CourseOffering)-[:OF_SUBJECT]->(pre)
        | {term: o2.term_code,
           start: head([(o2)-[:IN_TERM]->(x:Term) | x.start_date]),
           outcome: coalesce(t2.grade_letter, t2.competency_outcome, t2.status)}]
       AS attempts
WHERE none(a IN attempts WHERE a.start < term.start_date
                           AND a.outcome IN ['A', 'B', 'C'])
OPTIONAL MATCH (s)-[:HAS_ENROLMENT]->(:Enrolment {enrolment_status: 'ACTIVE'})
               -[:IN_PROGRAM]->(prog:Program)
RETURN s.student_no AS student_no, prog.program_code AS program,
       subj.subject_code AS subject, term.term_code AS term,
       pre.subject_code AS unmet_prerequisite,
       CASE size(attempts) WHEN 0 THEN 'never attempted'
            ELSE reduce(h = '', a IN attempts |
                        h + CASE WHEN h = '' THEN '' ELSE ', ' END
                          + a.term + ' ' + a.outcome) END AS prereq_history,
       CASE WHEN prog IS NULL THEN 'No active enrolment'
            WHEN NOT EXISTS { (prog)-[:INCLUDES]->(subj) }
                 THEN 'Enrolled outside program'
            WHEN NOT EXISTS { (prog)-[:INCLUDES]->(pre) } THEN 'Program design gap'
            ELSE 'Progression control failure' END AS root_cause
ORDER BY root_cause, program, student_no, subject;
```

Listing 4: *Q4 (abridged)*

Result. Ten breaches involve eight students. Five students took a subject outside their program (an enrolment-advice issue). Three breaches share the same curriculum issue: PRG102 (Business) includes SUB-0004 but not its prerequisite SUB-0005 so the program itself guarantees a breach. Two were progression-control failures: students progressed after a D or a *not yet competent* result. One correction to PRG102 removes 30% of the breaches.



```
$ MATCH (s:Student)-[t:TOOK]->(o:CourseOffering)-[:OF_SUBJECT]->(subj:Subject), (c
```

Raw

nt_no	student	program	subject	term	status	unmet_prere	prerequisite_	root_cause
026007"	"Lucas Hartley"	"PRG101"	"SUB-0008"	"2026T3"	"COMPLETED "	"SUB-0001"	"2026T1 D"	"Enrolled outside program"
026001"	"Tali Nguyen"	"PRG102"	"SUB-0007"	"2026T4"	"ENROLLED"	"SUB-0002"	"never attempted"	"Enrolled outside program"
026005"	"Ethan Brady"	"PRG103"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Enrolled outside program"
026003"	"Mia Fletcher"	"PRG104"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Enrolled outside program"
026014"	"Oscar Delaney"	"PRG104"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Enrolled outside program"

Started streaming 10 records after 1,743 ms and completed after 1,842 ms.

nt_no	student	program	subject	term	status	unmet_prere	prerequisite_	root_cause
026001"	"Tali Nguyen"	"PRG102"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Program design gap"
026011"	"Sienna Marsh"	"PRG102"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Program design gap"
026017"	"Isla Moreau"	"PRG102"	"SUB-0004"	"2026T3"	"ENROLLED"	"SUB-0005"	"never attempted"	"Program design gap"
026020"	"Malik Traore"	"PRG101"	"SUB-0004"	"2026T4"	"ENROLLED"	"SUB-0005"	"2026T3 NYC"	"Progressi on control failure"
026014"	"Oscar Delaney"	"PRG104"	"SUB-0008"	"2026T2"	"COMPLETED "	"SUB-0001"	"2026T1 D"	"Progressi on control failure"

Started streaming 10 records after 1,743 ms and completed after 1,842 ms.

Screenshot 5: *Q4 result in Neo4j Desktop (all ten rows, shown in two parts)*

6 Centrality Analysis (Task 4)

6.1 Use case and projection

The Head of Vocational Education needs a key-person risk register: whose absence would most affect teaching and communication? The delivery network was modelled as an undirected network with 65 nodes and 136 links. Priya Raman is both a student and a staff member, but a single node nonetheless. The nodes are Staff, CourseOffering, and Student, which are connected by the LEADS, TOOK, and SUPERVISES relationships. The seven GDS algorithms were run with the default settings (gds_centrality.txt; Neo4j, Inc., 2026b; Needham & Hodler, 2019): degree, PageRank and ArticleRank (an endorsement from well-connected neighbours), eigenvector and three path-based measures - betweenness, closeness and harmonic centrality.

6.2 Evaluation design

Since intuition could not pick among the seven scores, each was tested against two other factors that indicated what would probably happen if a person were lost. **G1,** harm done to the network: we found the global efficiency loss when a node was removed for all 65 of them (Latora & Marchiori, 2001). **G2,** the teaching load: the number of different students that a staff member teaches. The amount of

agreement was assessed using Spearman's ρ and Kendall's τ, along with 95% bootstrap confidence intervals and precision@3 on the staff ranking. Robustness to incomplete records was tested by randomly deleting 10% of TOOK relationships 500 times and comparing the perturbed staff ranking with the original.

6.3 Results

The algorithms belong to two families (Figure 7c). Only the path-based metrics, across all nodes, predict network damage: closeness (ρ = 0.93, 95% CI 0.86-0.97), betweenness (0.93, 0.86-0.96) and harmonic (0.90, 0.80-0.97), against 0.29-0.49 for PageRank, ArticleRank, degree and eigenvector, whose intervals do not overlap (Figure 7a). On the ten staff alone the algorithms have scores ρ = 0.82-0.92, but the intervals are too broad to differentiate them. Here, betweenness, PageRank, and ArticleRank all perfectly recover the genuine top three (precision@3 = 1.0). Degree and eigenvector best track teaching load (G2 ρ = 0.75 and 0.79); betweenness least (0.61). The gap is not surprising: the efficiency loss and the path-based measures both depend on shortest paths, whereas degree-type scores primarily reflect class sizes. Of the three damping factors examined (0.50, 0.85, 0.95; C7), PageRank's staff top three changed only at 0.95.

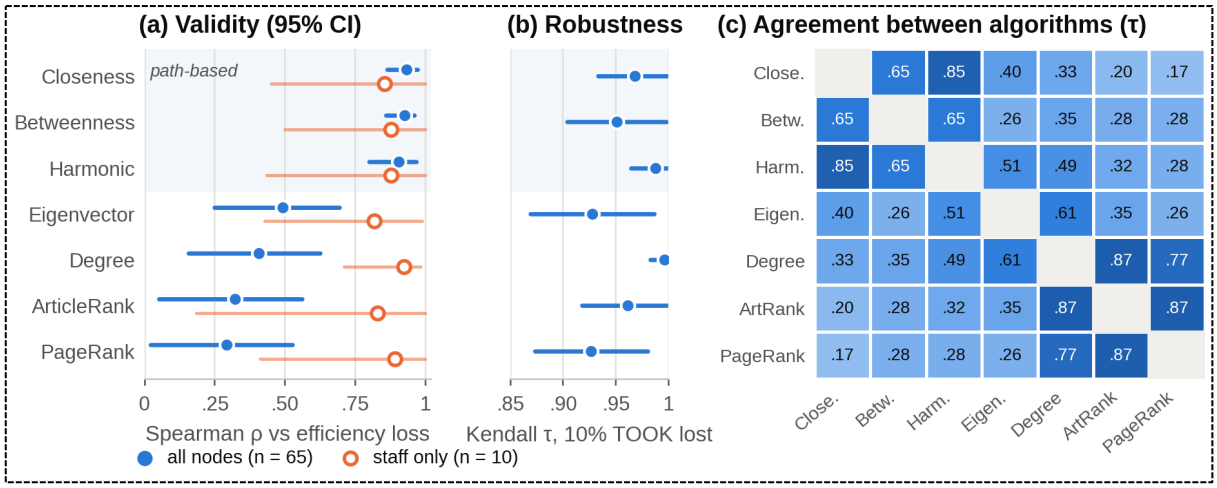


Figure 7: *Centrality algorithms compared*

(a) Agreement with network damage, all nodes and staff only, with 95% bootstrap intervals. (b) Rank stability when 10% of TOOK relationships are missing (mean ± SD, 500 runs). (c) Kendall τ between algorithms.

Two more discoveries are important in practice. The network is almost bipartite (offerings on one side, people on the other), its smallest and largest eigenvalues almost cancel (|λmin| / λmax = 0.98) and GDS needed 227 iterations to converge (C4). Thus, eigenvector centrality did not converge after the default 20 iterations of GDS. PageRank also reached its 20-iteration limit but its scores matched those of our replica to four decimal places. The most robust measure is degree (τ = 0.996), but it is also the coarsest one: five out of ten staff members are tied at degree 3. Harmonic is nearly as robust (0.988) with no ties. To summarise, the model is meaningful. If Priya's student role were a separate node like in Model A, her betweenness would decrease from 3rd to 5th.

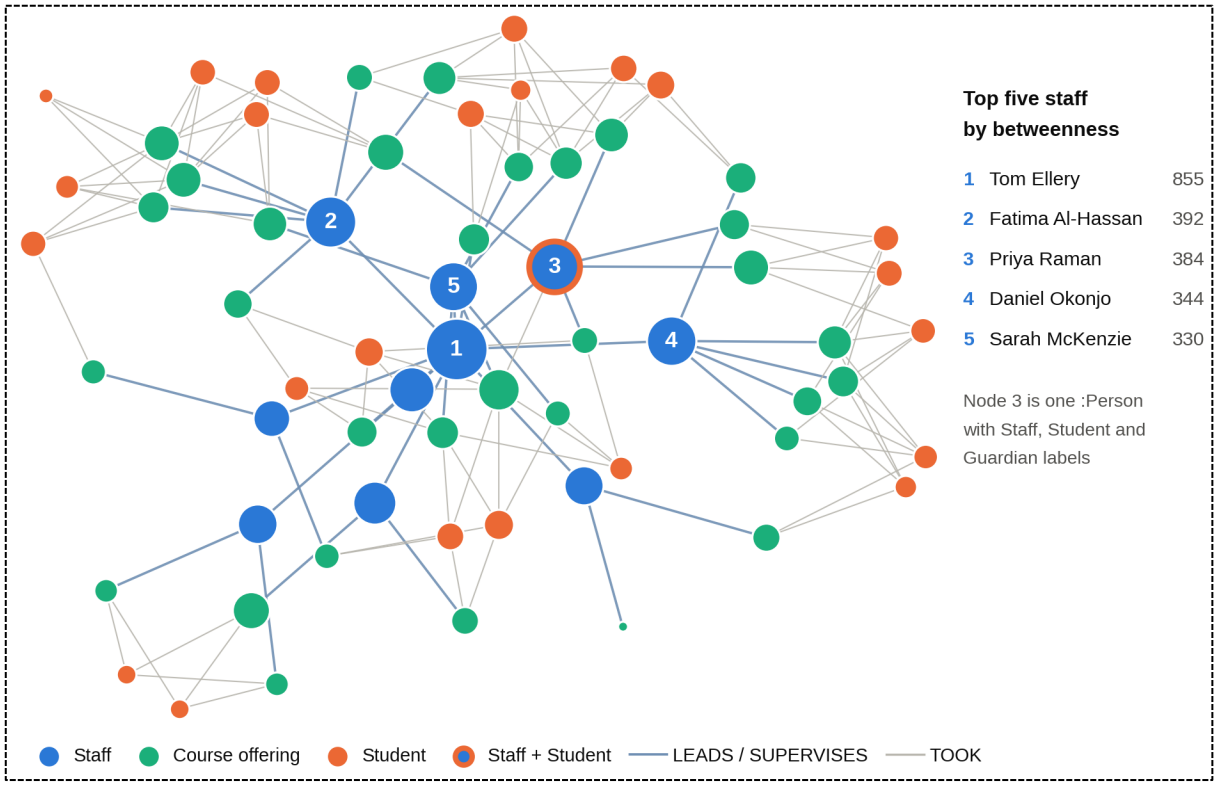


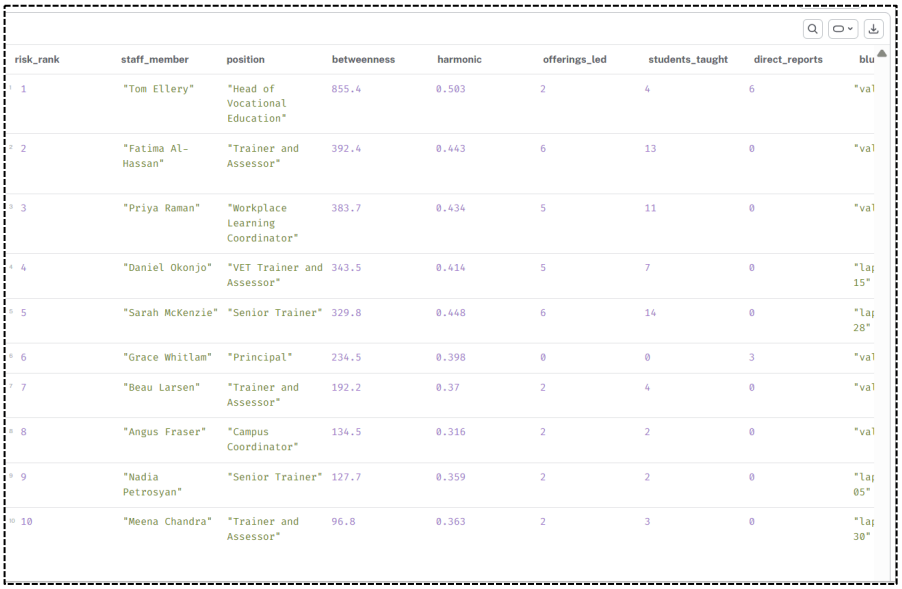
Figure 8: *The delivery network*
Node area is proportional to betweenness; the top five staff are numbered by rank.

6.4 Recommendation

We propose betweenness as the main key-person metric because it is scalable with the GDS sampling option, identifies the top three most important people and accurately predicts network damage. Harmonic centrality is the cross-check needed if some records are missing, or the network splits (when, say, a new campus opens). Degree is the best tool for balancing workload. PageRank, ArticleRank and eigenvector answer an endorsement question that this network does not ask. The scores are written back to the database and combined with the compliance data in a single query (C9):

Rank	Staff member	Betweenness	Harmonic	Offerings led	Students taught	Reports	Blue card
1	Tom Ellery, Head of Vocational Education	855.4	0.503	2	4	6	valid
2	Fatima Al-Hassan, Trainer and Assessor	392.4	0.443	6	13	0	valid
3	Priya Raman, Workplace Learning Coordinator	383.7	0.434	5	11	0	valid
4	Daniel Okonjo, VET Trainer and Assessor	343.5	0.414	5	7	0	lapses 15 Oct
5	Sarah McKenzie, Senior Trainer	329.8	0.448	6	14	0	lapses 28 Nov

Table 6: *Key-person risk register (top five by betweenness)*



	risk_rank	staff_member	position	betweenness	harmonic	offerings_led	students_taught	direct_reports	blu
1	1	"Tom Ellery"	"Head of Vocational Education"	855.4	0.503	2	4	6	"val
2	2	"Fatima Al-Hassan"	"Trainer and Assessor"	392.4	0.443	6	13	0	"val
3	3	"Priya Raman"	"Workplace Learning Coordinator"	383.7	0.434	5	11	0	"val
4	4	"Daniel Okonjo"	"VET Trainer and Assessor"	343.5	0.414	5	7	0	"lap 15"
5	5	"Sarah McKenzie"	"Senior Trainer"	329.8	0.448	6	14	0	"lap 28"
6	6	"Grace Whitlam"	"Principal"	234.5	0.398	0	0	3	"val
7	7	"Beau Larsen"	"Trainer and Assessor"	192.2	0.37	2	4	0	"val
8	8	"Angus Fraser"	"Campus Coordinator"	134.5	0.316	2	2	0	"val
9	9	"Nadia Petrosyan"	"Senior Trainer"	127.7	0.359	2	2	0	"lap 05"
10	10	"Meena Chandra"	"Trainer and Assessor"	96.8	0.363	2	3	0	"lap 30"

Screenshot 6: *C9 key-person register in Neo4j Desktop*

Tom Ellery is the single point of failure: six trainers report to him and removing him costs 12% of the network's efficiency, 2.5 times the next person's loss. Two of the top five also have lapsing blue cards.

7 Similarity Analysis (Task 5)

7.1 Use case and algorithms

Thirteen of the 24 students have no work placement. The placement officer wants to see where their most similar placed peers were successful, to decide which host employer fits each of them best. Similarity can be defined in various ways so we compared nine variants (`gds_similarity.txt`): GDS NodeSimilarity with Jaccard, overlap and cosine metrics on two bipartite projections - *what students study* (Student to Subject, 24 + 8 nodes) and *whom they study with* (Student to CourseOffering) - and KNN with cosine, Pearson and Euclidean metrics on *how well they perform*: an eight-value vector of grade points per subject built in Cypher. KNN was made deterministic (`concurrency: 1`, `randomSeed`). Because GDS KNN is approximate, the recommendation step asks for all 23 neighbours so every pair is scored.

7.2 Evaluation design

Each variant was tested in three ways.

What it measures: the share of each student's three closest peers in the same program or campus, relative to a random baseline.

What it buys: a leave-one-out test on the 11 placed students in which one student's placement was hidden and employers were ranked by the similarity-weighted votes of the three most similar placed peers, either from all students (*similarity only*) or from the same program (*within program*). Results were evaluated by hit rate and mean reciprocal rank (MRR) against random, popularity and program-rule baselines.

Whether it is stable: how often the third and fourth nearest peers are tied, which makes top-three membership arbitrary.

7.3 Results

Figure 9c shows the three definitions identify different peers: subject-based and class-based neighbours only share 10-15% of their top-three peers. Class-based similarity mostly picks up campus (86% same-campus peers vs. 19% by chance), while none of the variants captures programme strongly (26-35% vs. 22%). But similarity alone was no better than chance at placement matching (MRR 0.14-0.35 compared with 0.41 for random ranking). In a programme, 9 out of 11 students had the real employer identified

in the top three by each variant, the same as the programme rule baseline. The best variant, KNN-Euclidean, achieved an MRR of 0.61 compared to 0.64 for the rule. With n = 11 these differences are not significant, nor is the programme filter's gain for KNN-Euclidean (hit rate 6/11 to 9/11; Fisher's exact p = 0.36; Wilson score intervals overlap).

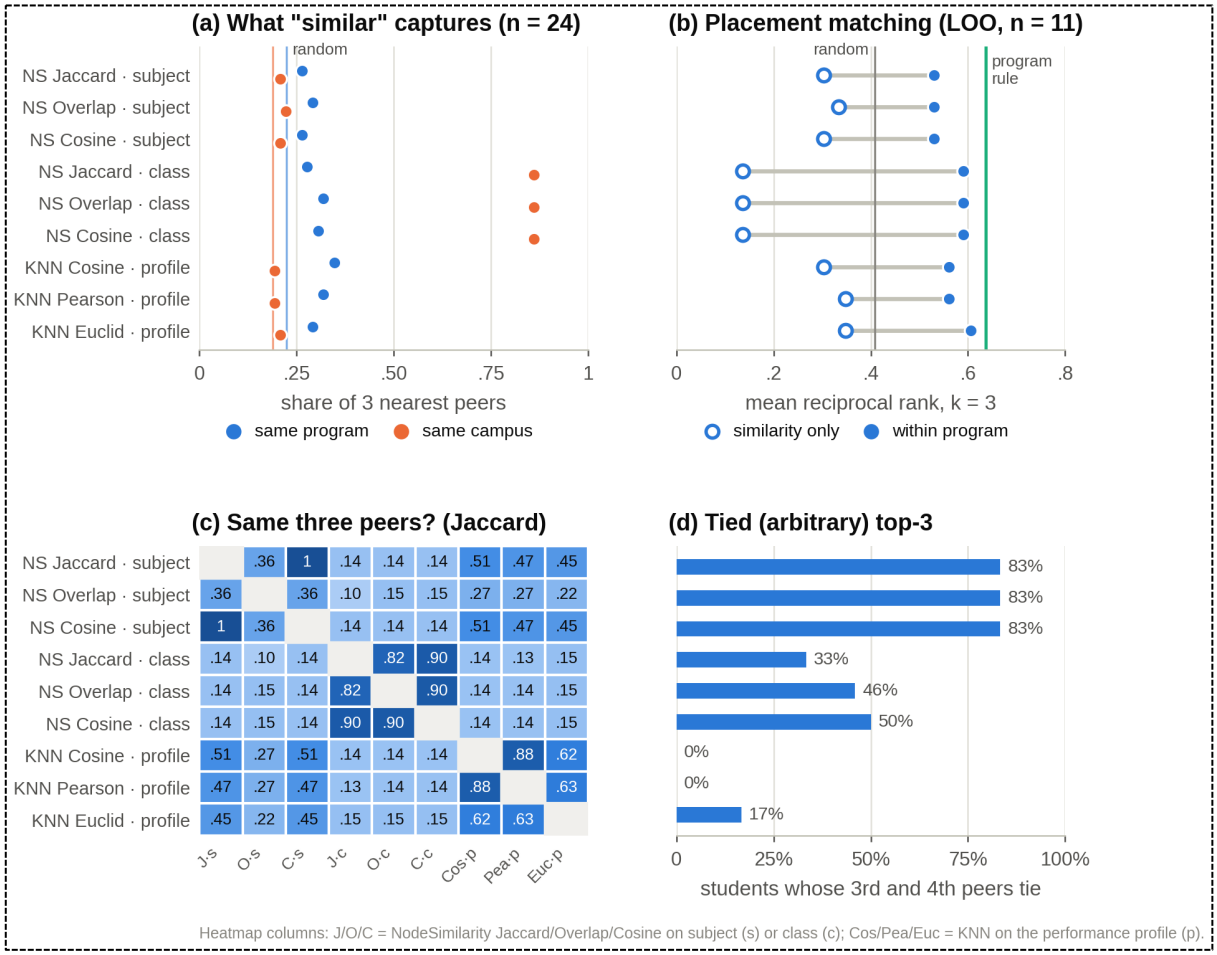


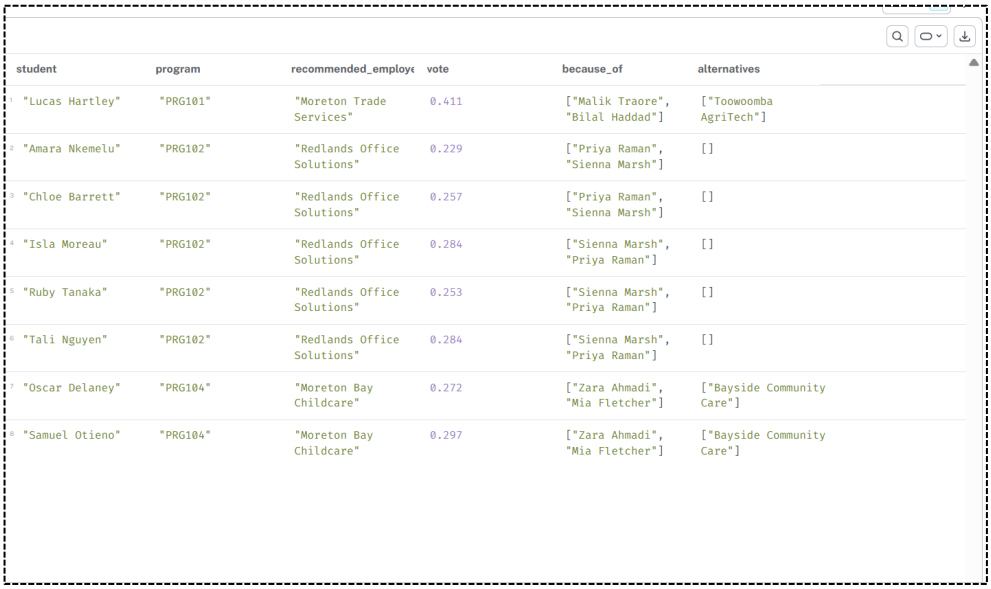
Figure 9: *Similarity variants compared*

(a) What each similarity notion captures. (b) Leave-one-out placement matching. (c) Overlap of top-three peer sets between variants. (d) Share of students whose top three is decided by a tie.

Stability distinguishes variants better than accuracy does. NodeSimilarity on eight subjects results in ties for 83% of students so GDS's tie-breaking, not the data, decides who counts as a peer. The KNN profile variants tie for 0-17%. That was confirmed by the Neo4j run: GDS picked a different peer from our replica with the same score for at least four students. The KNN lists agreed except at ties and for one student where approximate KNN missed a neighbour. The study also identified a cold-start gap that no similarity method can overcome: all five PRG103 (Engineering) students are unplaced and two hosts, including Ipswich Precision Metals, the only manufacturing host, have never hosted a student.

7.4 Recommendation

We propose a hybrid approach: use programme as a hard filter, then apply KNN with Euclidean distance to the performance profile to rank employers within the programme. It is deterministic, rarely tied, explainable by named peers, and ties the best baseline for hits. It is the best variant at k=3 and at k=1 it is one of four that edge ahead of the rule (MRR 0.68 vs 0.64: one student ranked first instead of second). The other three are subject-based NodeSimilarity variants in which ties determine peers. It recommends Moreton Trade Services, Redlands Office Solutions, and Moreton Bay Childcare for the eight unplaced PRG101, PRG102, and PRG104 students, respectively. The PRG103 cohort's content rule is 'Engineering meets Ipswich Precision Metals'. Similarity can not suggest what no one has done.



	student	program	recommended_employe	vote	because_of	alternatives
1	"Lucas Hartley"	"PRG101"	"Moreton Trade Services"	0.411	["Malik Traore", "Bilal Haddad"]	["Toowoomba AgriTech"]
2	"Amara Nkemelu"	"PRG102"	"Redlands Office Solutions"	0.229	["Priya Raman", "Sienna Marsh"]	[]
3	"Chloe Barrett"	"PRG102"	"Redlands Office Solutions"	0.257	["Priya Raman", "Sienna Marsh"]	[]
4	"Isla Moreau"	"PRG102"	"Redlands Office Solutions"	0.284	["Sienna Marsh", "Priya Raman"]	[]
5	"Ruby Tanaka"	"PRG102"	"Redlands Office Solutions"	0.253	["Sienna Marsh", "Priya Raman"]	[]
6	"Tali Nguyen"	"PRG102"	"Redlands Office Solutions"	0.284	["Sienna Marsh", "Priya Raman"]	[]
7	"Oscar Delaney"	"PRG104"	"Moreton Bay Childcare"	0.272	["Zara Ahmadi", "Mia Fletcher"]	["Bayside Community Care"]
8	"Samuel Otieno"	"PRG104"	"Moreton Bay Childcare"	0.297	["Zara Ahmadi", "Mia Fletcher"]	["Bayside Community Care"]

Screenshot 7: *S9 placement recommendations in Neo4j Desktop*

8 Discussion and Limitations

Answers to the research questions

RQ1: The optimised model is 23% cheaper on the workload and as much as 37% cheaper on single queries, and the ablation shows that folding and materialisation provide most of the gain.

RQ2: The graph answered all the stakeholder queries. The findings from SQL were the same and the graph expressed the recursion, cycles and root-cause reasoning more directly than SQL.

RQ3: The algorithms were picked based on facts and not gut feelings. Path-based centrality is used for network damage. The only suitable location for similarity is inside a program filter.

Links to the course material

In Week 7, we were introduced to graph databases using the example of books and authors with many-to-many relationships. SQL requires a join table and a 3-way join. Cypher needs a pattern. Academically, our reified relationships and the Q4 prerequisite chain are the same thing (Charles Darwin University, 2026; Lemahieu et al., 2018). The course describes NoSQL databases as BASE, but Neo4j is an exception because every statement runs in an ACID transaction; hence, it follows the Week 9 principles in the design. If the unbatched import statement fails, the whole thing is rolled back. Uniqueness constraints and MERGE handle the consistency obligations the lecture imposes on the developer. To prevent the lost-update problem discussed in Week 9, and because the default isolation level in Neo4j is read committed, we must lock the invoice before reading its amount when processing two simultaneous payments against it (Neo4j, Inc., 2026c). Concurrency was not tested. Week 8 is the key to deployment. Our scripts include the reporting date and keys so we can validate the results. An application would pass them as parameters via the Bolt driver, similar to a JDBC prepared statement, allowing reuse of the query plan. Week 10 includes the following phases. The CSV export is the initial stage of an ETL process. Q3 is a roll-up of invoicing data to campus level. The in-memory GDS projection is a dependent data mart which provides a tailored replica of the operational graph for analysis. The graph is not a data warehouse because it holds current balances, not history.

Limitations

First, the collection is tiny and composed (600 rows). This implies that each outcome can be verified but it also decreases the statistical power. The placement test with 11 cases and the intervals of staff-level centrality are unable to distinguish between close rivals. Second, costs are logical accesses counted on a replica, not Neo4j database hits. It does not take into account caching or runtime so the result is the order of the variations and not the precise numbers. The PROFILE script may verify that. Third, after the 20 rounds, the eigenvector scores for GDS were different from the replicas' in the third decimal place but they still rated the staff in the same order. Fourth, materialised amounts may become stale if payment is loaded without re-running section 7.2. You can solve this with a scheduled update or an APOC trigger. Fifth, the Community Edition cannot express NOT NULL or cardinality restrictions, therefore validation queries make sure they are observed. Lastly, these metrics would profile actual people. Key-person scores are only to be used for succession planning and not for performance management, in line with the school's privacy policy.

Future work

The ablation ranking would be evaluated on a large scale in a synthetic school that was 10-100 times larger than the original. Content-based matching can be employed to address a cold-start cohort by providing more precise placement information, such as the location of the host and the number of units required by each host. Versioned relationships would facilitate audit enquiries that are conducted at specific points in time.

9 Conclusion

The Industry School's relational database has been thoroughly and verifiably migrated to Neo4j. We developed two models using the same loader to assess the optimisations and an ablation study to identify the worthwhile ones. The resulting graph allows the school to address regulatory, budgetary and curricular challenges with a single understandable query. The analytics were chosen based on evidence: program-filtered KNN for placement matching, and betweenness with harmonic as a cross-check for key-person risk. The same approach: model, appraise, then suggest - applies to other data and to the questions the institution still has to answer.

References

Australian Skills Quality Authority. (2025). *Standards for RTOs*. https://www.asqa.gov.au/for-providers/standards-for-RTOs

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Latora, V., & Marchiori, M. (2001). Efficient behavior of small-world networks. *Physical Review Letters, 87*(19), Article 198701. https://doi.org/10.1103/PhysRevLett.87.198701

Lemahieu, W., vanden Broucke, S., & Baesens, B. (2018). *Principles of database management: The practical guide to storing, managing and analyzing big and small data*. Cambridge University Press.

Needham, M., & Hodler, A. E. (2019). *Graph algorithms: Practical examples in Apache Spark and Neo4j*. O'Reilly Media.

Neo4j, Inc. (2026a). *Neo4j Cypher manual* (Version 5). https://neo4j.com/docs/cypher-manual/current/

Neo4j, Inc. (2026b). *Neo4j Graph Data Science library manual*. https://neo4j.com/docs/graph-data-science/current/

Neo4j, Inc. (2026c). *Concurrent data access*. In *Neo4j operations manual* (Version 5). https://neo4j.com/docs/operations-manual/current/database-internals/concurrent-data-access/

Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: New opportunities for connected data* (2nd ed.). O'Reilly Media.

Working with Children (Risk Management and Screening) Act 2000 (Qld).

Individual Contributions

Work was planned and tracked on a shared Trello board: one list per task; cards with checklists, owners, due dates, and task labels; Butler rules that moved a card to Review when its checklist was complete; and a weekly review card recording decisions. The board export and decision log are in the repository.

Cynthia Ebele Ikegbunam (Group Leader)

Coordinated the project and the Trello board, set weekly milestones and ran the review meetings. Wrote the Introduction, the Assessment 2 revisions (Table 1) and the Conclusion; developed use case Q3 and the corrected debt query.

Md Adnan Abir

Designed both graph models and the conversion rules (Section 2), wrote import.txt and the baseline loader, built the reconciliation and the modelling experiment (Section 4) and produced Figures 1-6.

Chepngenoh S Chepkwony

Developed use cases Q1, Q2 and Q4, tested every query against the expected results and prepared the PROFILE script and Listings 1-4 (Section 5).

Najmus Sakeeb

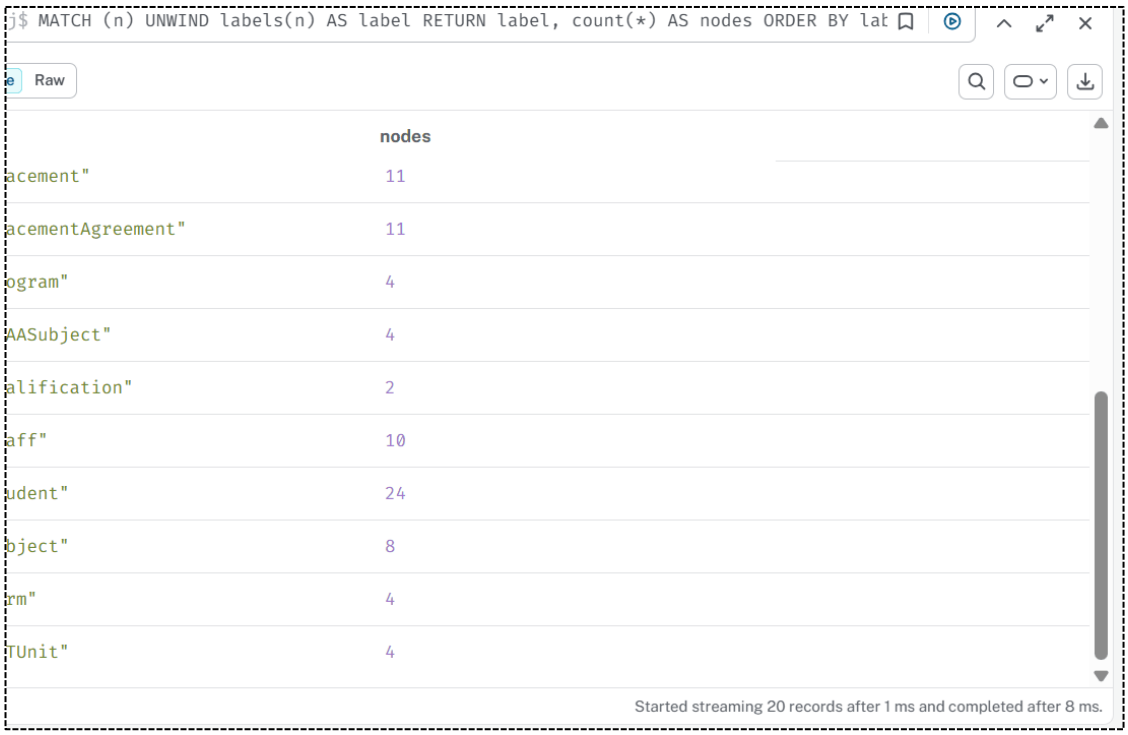
Designed and ran the centrality and similarity analyses (Sections 6-7), including the ground-truth and leave-one-out evaluations, and produced Figures 7-9 and the GDS scripts; integrated and proofread the final report.

Appendix A - Neo4j run evidence

The remaining screenshots from the Neo4j run (Neo4j Desktop 2.2.1, Neo4j 2026.09.0, GDS 2026.09.0, 14 October 2026), in the order the statements were run. Each caption names the script and statement number.



Screenshot A1: *import.txt 8.1 - nodes per label (part 1 of 2)*



Screenshot A2: *import.txt 8.1 - nodes per label (part 2 of 2)*

```
neo4j$ MATCH ()-[r]->() RETURN type(r) AS relationship, count(*) AS rel
```

Table Raw

	relationship	relationships
1	"AT_CAMPUS"	58
2	"AWARDED"	2
3	"BILLED_TO"	42
4	"FOR_TERM"	42
5	"GOVERNED_BY"	11
6	"GUARDIAN_OF"	16
7	"HAS_ENROLMENT"	26
8	"HAS_LINE"	73
9	"HOLDS_ACCOUNT"	22
10	"HOSTED_BY"	11

Screenshot A3: *import.txt 8.2 - relationships per type (part 1 of 3)*

Table Raw

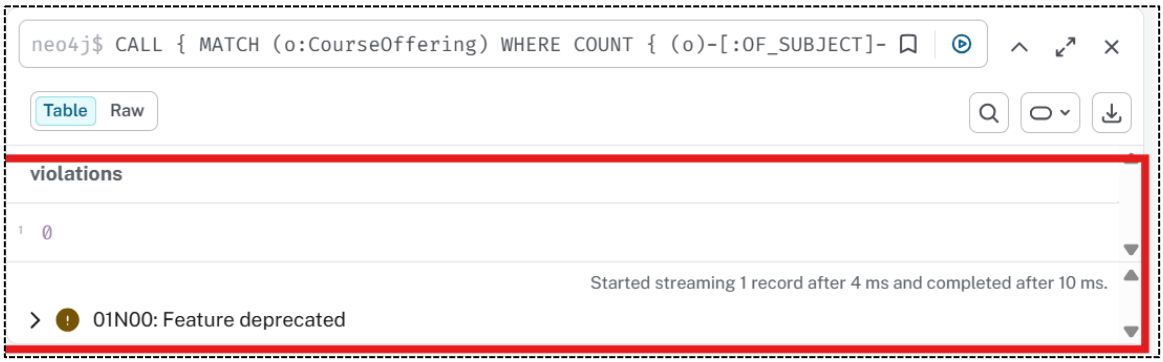
	relationship	relationships
10	"HOSTED_BY"	11
11	"INCLUDES"	12
12	"IN_PROGRAM"	26
13	"IN_TERM"	32
14	"LEADS"	32
15	"NEXT_TERM"	3
16	"OF_SUBJECT"	32
17	"RAISED_FOR"	42
18	"REQUIRES"	3
19	"SETTLES"	33

Screenshot A4: *import.txt 8.2 - relationships per type (part 2 of 3)*

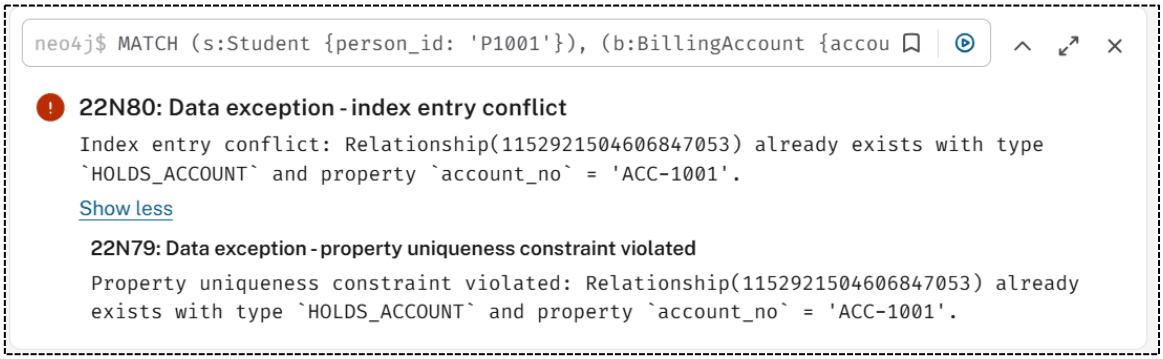
Table Raw

	relationship	relationships
13	"IN_TERM"	32
14	"LEADS"	32
15	"NEXT_TERM"	3
16	"OF_SUBJECT"	32
17	"RAISED_FOR"	42
18	"REQUIRES"	3
19	"SETTLES"	33
20	"SUPERVISES"	9
21	"TOOK"	95
22	"UNDERTAKES"	11

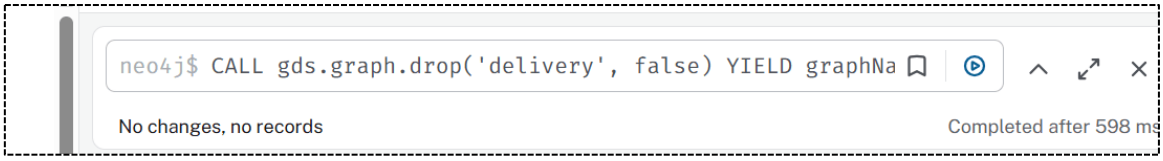
Screenshot A5: *import.txt 8.2 - relationships per type (part 3 of 3)*



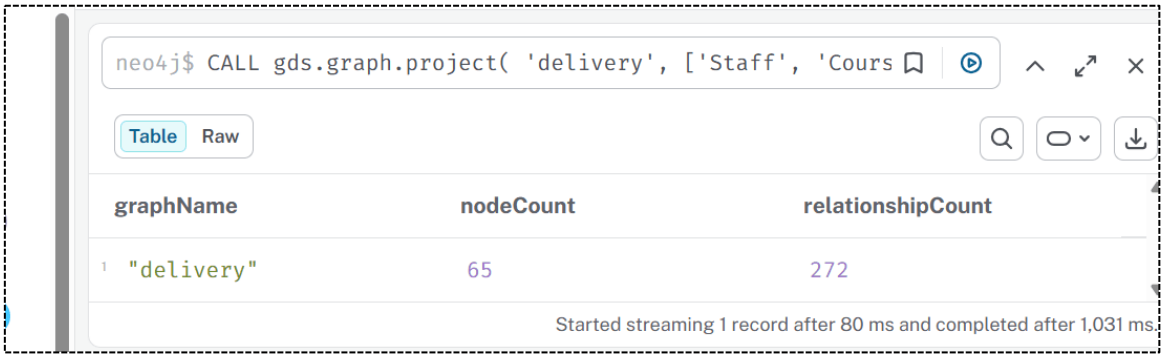
Screenshot A6: *import.txt 8.7 - structural integrity check (0 violations)*



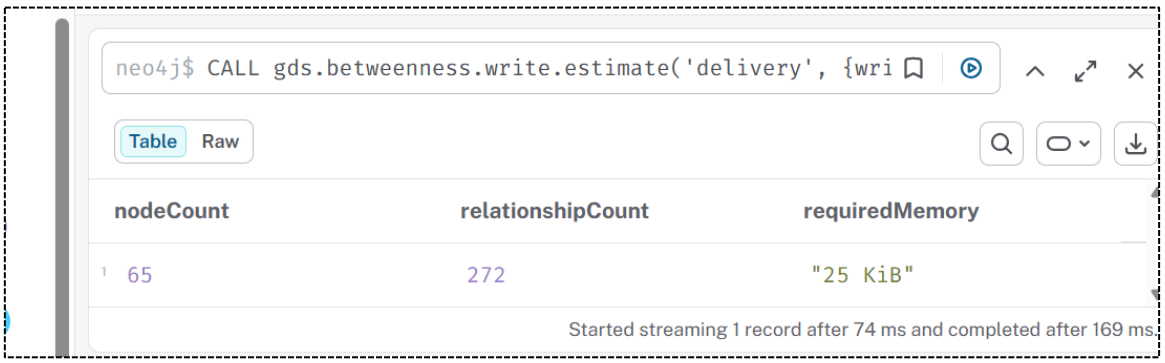
Screenshot A7: *import.txt 8.8 - the uniqueness constraint rejects a second holder for ACC-1001*



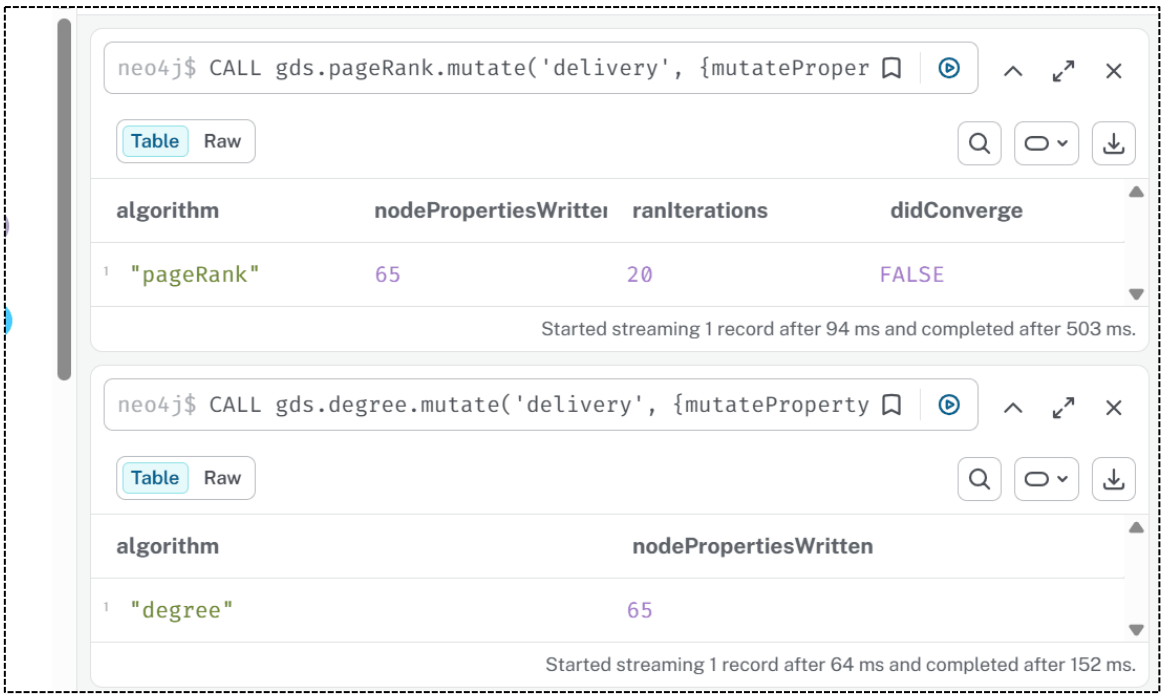
Screenshot A8: *gds_centrality.txt C0 - drop any earlier projection*



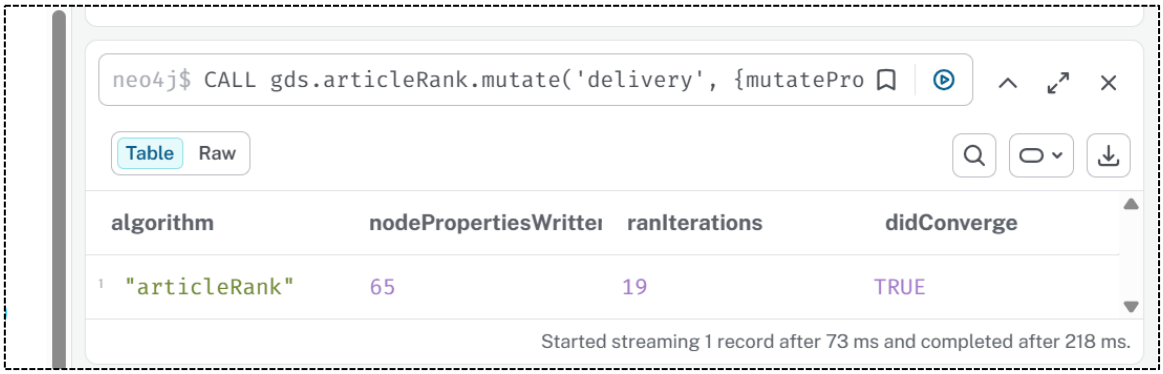
Screenshot A9: *gds_centrality.txt C1 - delivery network projected (65 nodes, 272 relationships)*



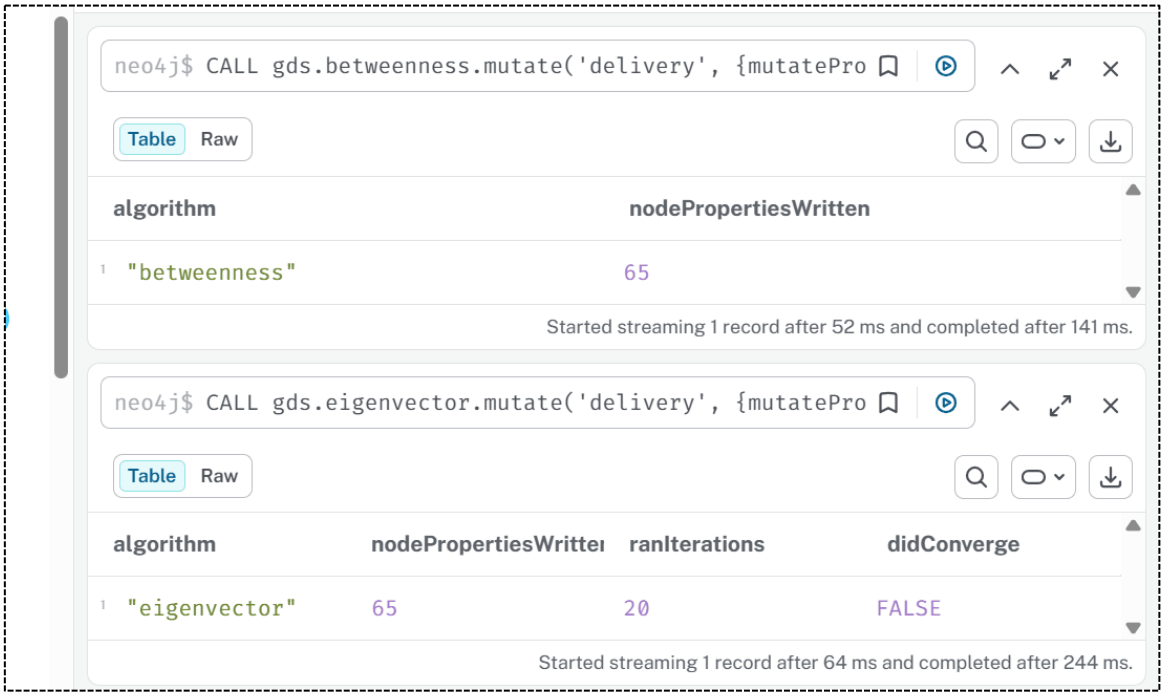
Screenshot A10: *gds_centrality.txt C2 - memory estimate*



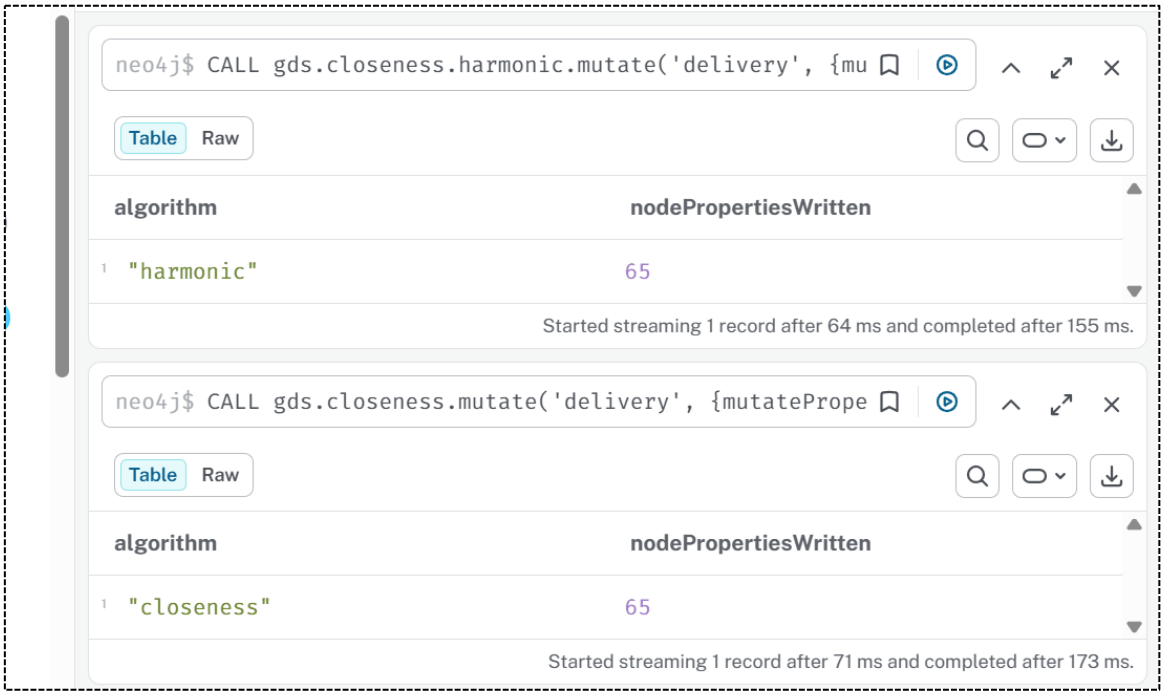
Screenshot A11: *gds_centrality.txt C3 - PageRank and degree*



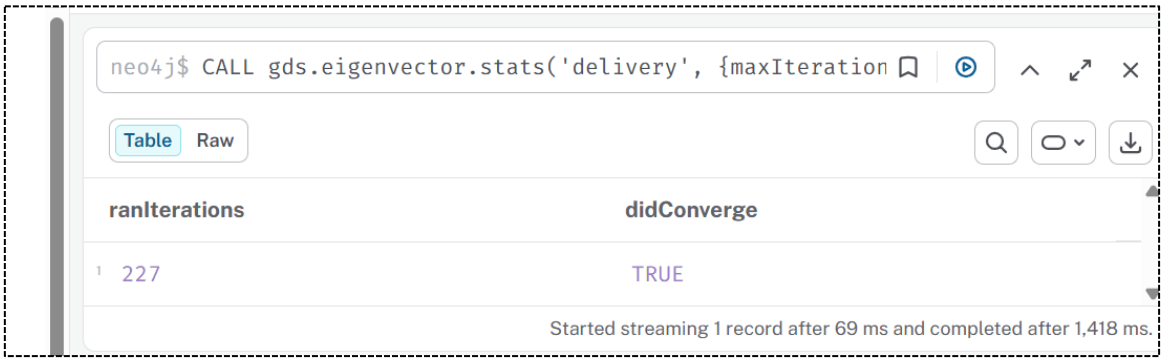
Screenshot A12: *gds_centrality.txt C3 - ArticleRank*



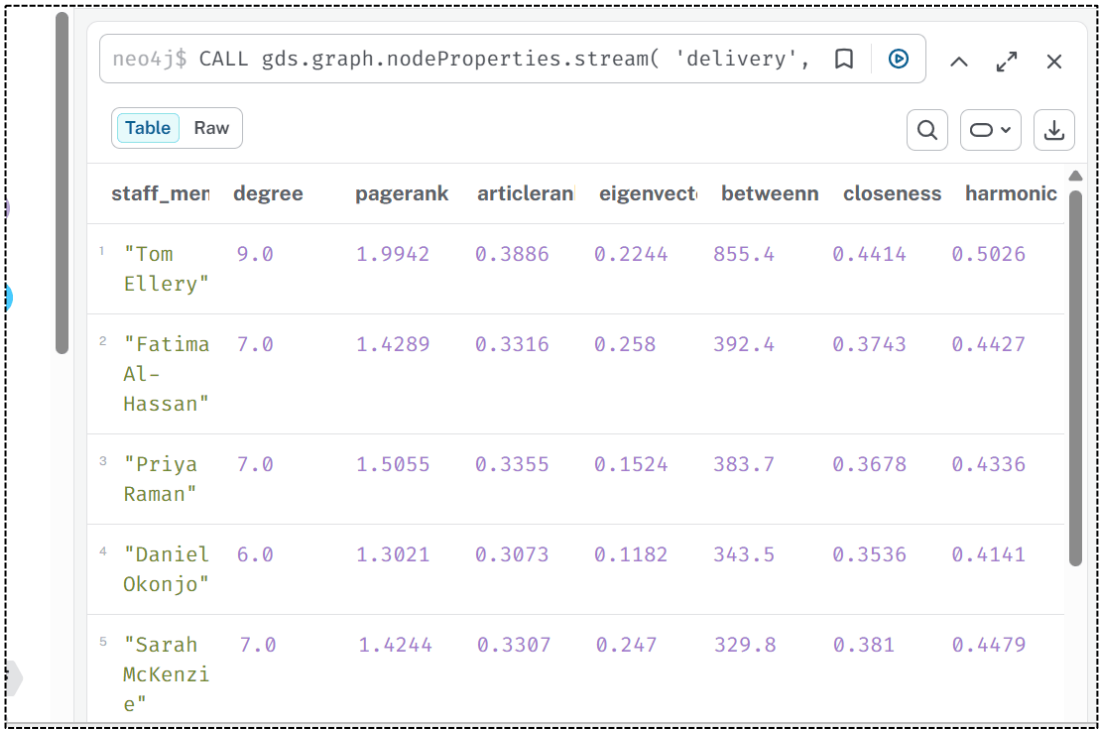
Screenshot A13: *gds_centrality.txt C3 - betweenness and eigenvector*



Screenshot A14: *gds_centrality.txt C3 - harmonic and closeness*



Screenshot A15: *gds_centrality.txt C4 - eigenvector converges after 227 iterations*

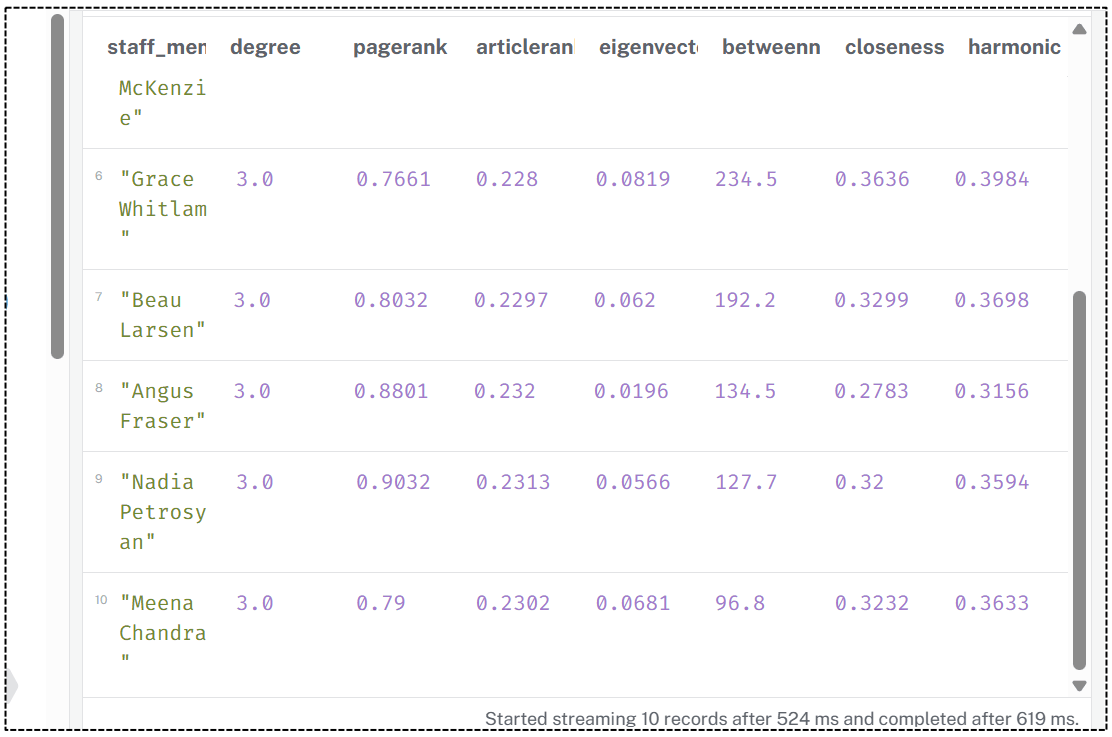


```
neo4j$ CALL gds.graph.nodeProperties.stream( 'delivery',
```

Table | Raw

	staff_mem	degree	pagerank	articleran	eigenvect	betweenn	closeness	harmonic
1	"Tom Ellery"	9.0	1.9942	0.3886	0.2244	855.4	0.4414	0.5026
2	"Fatima Al-Hassan"	7.0	1.4289	0.3316	0.258	392.4	0.3743	0.4427
3	"Priya Raman"	7.0	1.5055	0.3355	0.1524	383.7	0.3678	0.4336
4	"Daniel Okonjo"	6.0	1.3021	0.3073	0.1182	343.5	0.3536	0.4141
5	"Sarah McKenzie"	7.0	1.4244	0.3307	0.247	329.8	0.381	0.4479

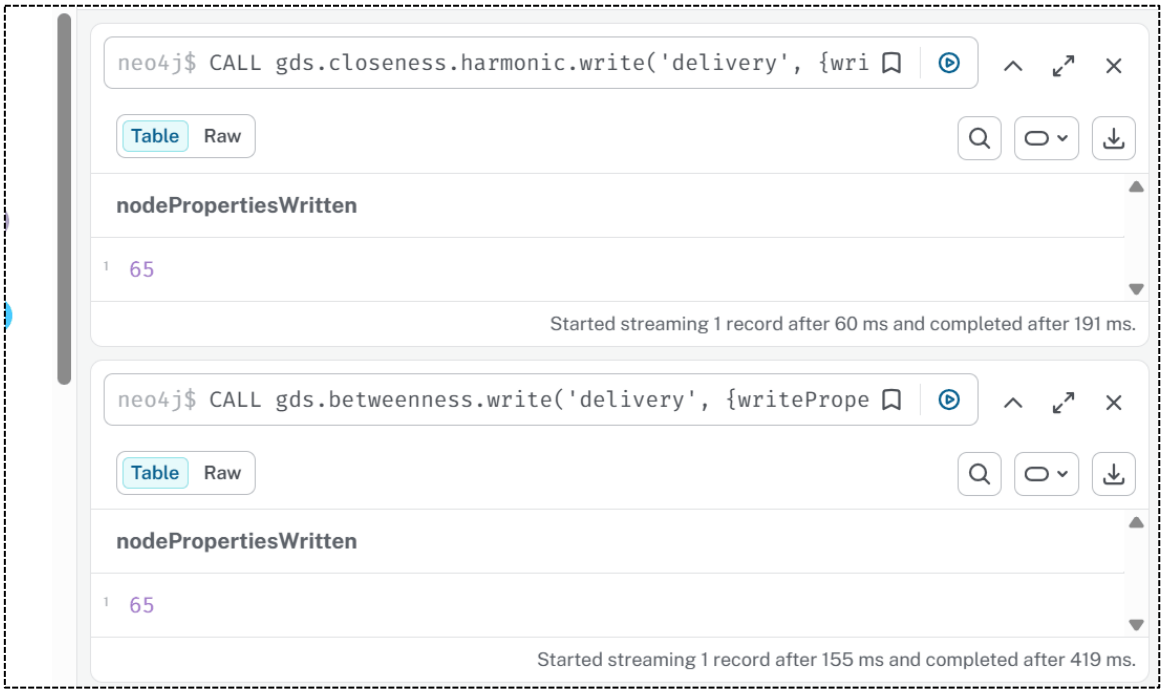
Screenshot A16: *gds_centrality.txt C5 - all seven scores for the ten staff (part 1 of 2)*



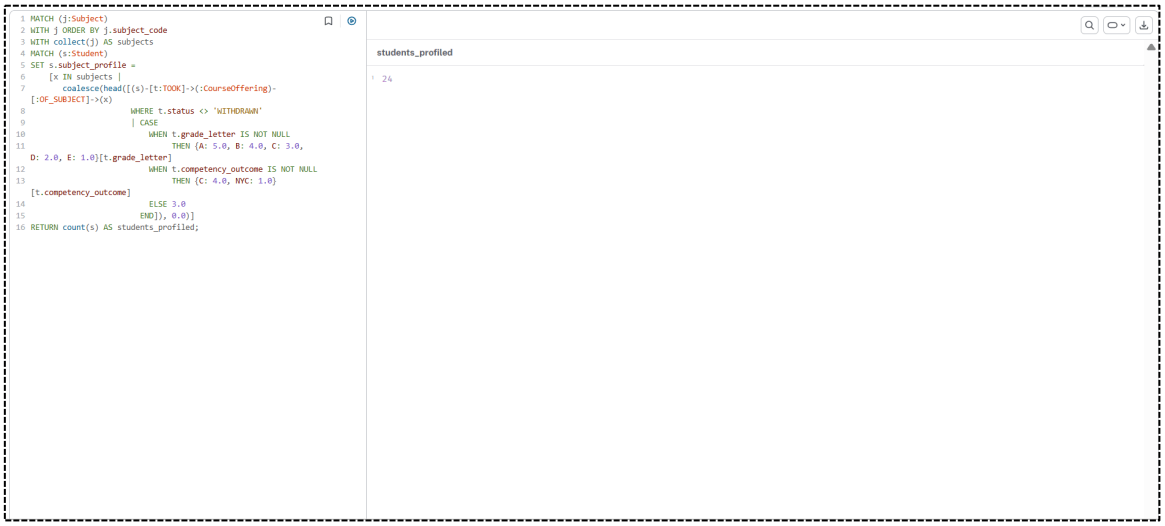
	staff_mem	degree	pagerank	articleran	eigenvect	betweenn	closeness	harmonic
	McKenzie"							
6	"Grace Whitlam"	3.0	0.7661	0.228	0.0819	234.5	0.3636	0.3984
7	"Beau Larsen"	3.0	0.8032	0.2297	0.062	192.2	0.3299	0.3698
8	"Angus Fraser"	3.0	0.8801	0.232	0.0196	134.5	0.2783	0.3156
9	"Nadia Petrosyan"	3.0	0.9032	0.2313	0.0566	127.7	0.32	0.3594
10	"Meena Chandra"	3.0	0.79	0.2302	0.0681	96.8	0.3232	0.3633

Started streaming 10 records after 524 ms and completed after 619 ms.

Screenshot A17: *gds_centrality.txt C5 - all seven scores for the ten staff (part 2 of 2)*



Screenshot A18: *gds_centrality.txt C8 - scores written back to the database*



Screenshot A19: *gds_similarity.txt S0 - subject performance profile for every student*

```
MATCH (s:Student)-[t:TOOK]->(:CourseOffering)-[:OF_SUBJECT]->(j:Subject)
WHERE t.status <> 'WITHDRAWN'
WITH DISTINCT s, j
WITH gds.graph.project('studentSubject', s, j,
        {sourceNodeLabels: ['Student'], targetNodeLabels: ['Subject']}) AS g
RETURN g.graphName AS graph, g.nodeCount AS nodes,
g.relationshipCount AS relationships;
```

graph	nodes	relationships
"studentSubject"	32	94

Screenshot A20: *gds_similarity.txt S2 - Student-Subject projection*

```
neo4j$ MATCH (s:Student) WITH gds.graph.project('studentProfile', s, null, {sourceNodeLabels: ['Student'], targetNodeLabels:
```

Table Raw

graph	nodes
"studentProfile"	24

Started streaming 1 record after 142 ms and completed after 333 ms.

Screenshot A21: *gds_similarity.txt S4 - performance-profile projection*

```
neo4j$ CALL gds.nodeSimilarity.stream('studentSubject', {similarityMetric: 'JACCARD', topK: 3}) YIELD node1, node2, simila
```

Table Raw

student	three_most_similar
"Aarav Chandra"	["Jayden Okafor (1.0)", "Amara Nkemelu (0.8)", "Noah Petrov (0.75)"]
"Amara Nkemelu"	["Freya Lindqvist (0.8)", "Lucas Hartley (0.8)", "Jayden Okafor (0.8)"]
"Bilal Haddad"	["Malik Traore (1.0)", "Layla Osman (1.0)", "Noah Petrov (0.75)"]
"Chloe Barrett"	["Zara Ahmadi (1.0)", "Amara Nkemelu (0.8)", "Ruby Tanaka (0.6)"]
"Diego Salazar"	["Ethan Brady (0.8)", "Hunter Kelly (0.6)", "Ruby Tanaka (0.6)"]
"Ethan Brady"	["Diego Salazar (0.8)", "Sienna Marsh (0.667)", "Tali Nguyen (0.667)"]
"Freya Lindqvist"	["Lucas Hartley (1.0)", "Amara Nkemelu (0.8)", "Noah Petrov (0.75)"]
"Hunter Kelly"	["Sienna Marsh (0.8)", "Ruby Tanaka (0.6)", "Jayden Okafor (0.6)"]
"Isla Moreau"	["Sienna Marsh (0.8)", "Bilal Haddad (0.6)", "Jayden Okafor (0.6)"]

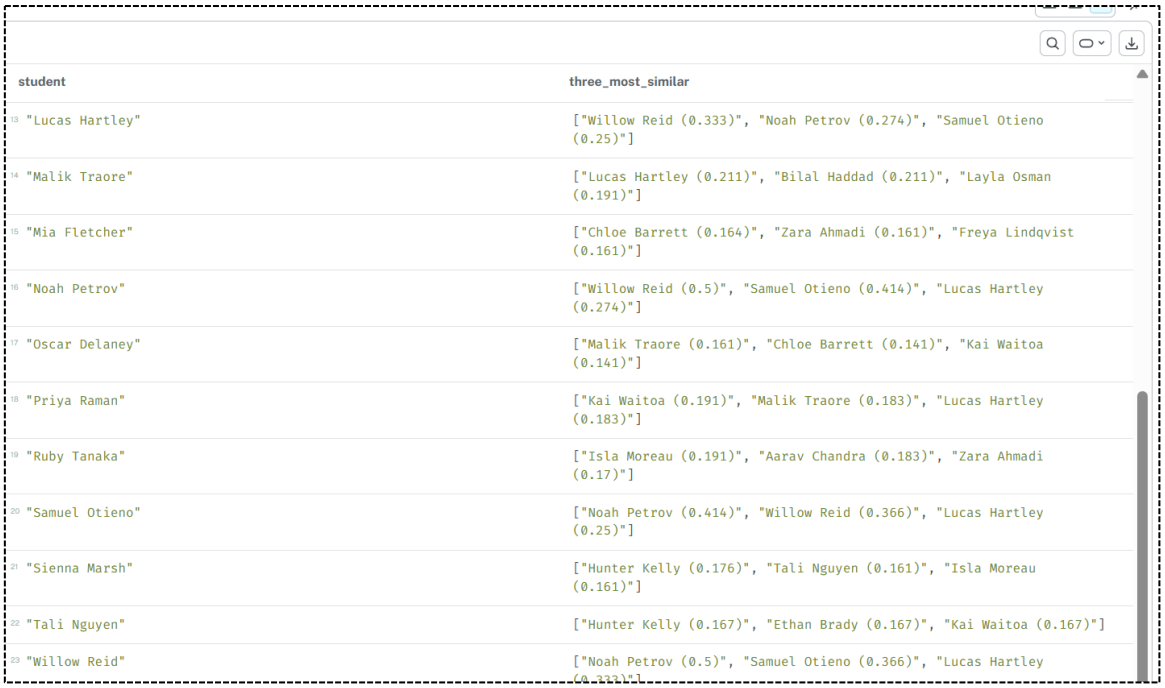
Started streaming 24 records after 347 ms and completed after 706 ms.

Screenshot A22: *gds_similarity.txt S5 - NodeSimilarity (Jaccard), three most similar peers*



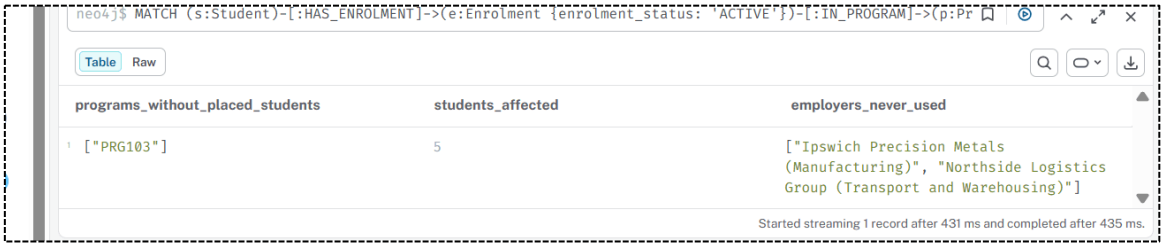
	student	three_most_similar
1	"Aarav Chandra"	["Jayden Okafor (0.274)", "Samuel Otieno (0.24)", "Amara Nkemelu (0.224)"]
2	"Amara Nkemelu"	["Freya Lindqvist (0.24)", "Aarav Chandra (0.224)", "Jayden Okafor (0.173)"]
3	"Bilal Haddad"	["Layla Osman (0.414)", "Willow Reid (0.24)", "Noah Petrov (0.232)"]
4	"Chloe Barrett"	["Zara Ahmadi (0.5)", "Amara Nkemelu (0.167)", "Mia Fletcher (0.164)"]
5	"Diego Salazar"	["Ethan Brady (0.205)", "Ruby Tanaka (0.164)", "Hunter Kelly (0.161)"]
6	"Ethan Brady"	["Diego Salazar (0.205)", "Tali Nguyen (0.167)", "Sienna Marsh (0.159)"]
7	"Freya Lindqvist"	["Amara Nkemelu (0.24)", "Samuel Otieno (0.224)", "Noah Petrov (0.211)"]
8	"Hunter Kelly"	["Kai Waitoa (0.29)", "Zara Ahmadi (0.187)", "Sienna Marsh (0.176)"]
9	"Isla Moreau"	["Ruby Tanaka (0.191)", "Aarav Chandra (0.183)", "Zara Ahmadi (0.17)"]
10	"Jayden Okafor"	["Aarav Chandra (0.274)", "Willow Reid (0.2)", "Noah Petrov (0.195)"]
11	"Kai Waitoa"	["Hunter Kelly (0.29)", "Priya Raman (0.191)", "Jayden Okafor (0.191)"]
12	"Layla Osman"	["Bilal Haddad (0.414)", "Noah Petrov (0.195)", "Samuel Otieno

Screenshot A23: *gds_similarity.txt S6 - KNN (Euclidean), three most similar peers (part 1 of 2)*



	student	three_most_similar
13	"Lucas Hartley"	["Willow Reid (0.333)", "Noah Petrov (0.274)", "Samuel Otieno (0.25)"]
14	"Malik Traore"	["Lucas Hartley (0.211)", "Bilal Haddad (0.211)", "Layla Osman (0.191)"]
15	"Mia Fletcher"	["Chloe Barrett (0.164)", "Zara Ahmadi (0.161)", "Freya Lindqvist (0.161)"]
16	"Noah Petrov"	["Willow Reid (0.5)", "Samuel Otieno (0.414)", "Lucas Hartley (0.274)"]
17	"Oscar Delaney"	["Malik Traore (0.161)", "Chloe Barrett (0.141)", "Kai Waitoa (0.141)"]
18	"Priya Raman"	["Kai Waitoa (0.191)", "Malik Traore (0.183)", "Lucas Hartley (0.183)"]
19	"Ruby Tanaka"	["Isla Moreau (0.191)", "Aarav Chandra (0.183)", "Zara Ahmadi (0.17)"]
20	"Samuel Otieno"	["Noah Petrov (0.414)", "Willow Reid (0.366)", "Lucas Hartley (0.25)"]
21	"Sienna Marsh"	["Hunter Kelly (0.176)", "Tali Nguyen (0.161)", "Isla Moreau (0.161)"]
22	"Tali Nguyen"	["Hunter Kelly (0.167)", "Ethan Brady (0.167)", "Kai Waitoa (0.167)"]
23	"Willow Reid"	["Noah Petrov (0.5)", "Samuel Otieno (0.366)", "Lucas Hartley

Screenshot A24: *gds_similarity.txt S6 - KNN (Euclidean), three most similar peers (part 2 of 2)*



```
neo4j$ MATCH (s:Student)-[:HAS_ENROLMENT]->(e:Enrolment {enrolment_status: 'ACTIVE'})-[:IN_PROGRAM]->(p:Pr
```

Table Raw

	programs_without_placed_students	students_affected	employers_never_used
1	["PRG103"]	5	["Ipswich Precision Metals (Manufacturing)", "Northside Logistics Group (Transport and Warehousing)"]

Started streaming 1 record after 431 ms and completed after 435 ms.

Screenshot A25: *gds_similarity.txt S10 - cold-start diagnostics*